

Sistema de transacciones financieras por voz en dispositivos móviles iOS

Ing. Alexis Geraldine Barniquez Piñero

Carrera de Especialización en Inteligencia Artificial

Director: Ing. Sergio Ivaldi

Jurados:

Jurado 1 (pertenencia)

Jurado 2 (pertenencia)

Jurado 3 (pertenencia)

Ciudad de Buenos Aires, junio de 2026

Resumen

En la presente memoria se desarrolla una prueba de concepto de una aplicación móvil nativa para iOS, capaz de procesar comandos de voz e interpretar intenciones transaccionales de forma 100 % local. Este sistema fue desarrollado para ser validado por el equipo de Identidad Digital del Banco Galicia Minorista.

Para su materialización, se utilizaron herramientas del ecosistema Apple como Swift, Xcode, la API AVFoundation y el framework Core ML.

Agradecimientos

Quiero dedicar un reconocimiento especial a quienes me acompañaron en este proceso. A mi madrina Engels, gracias por ser siempre mi mayor fuente de inspiración y un verdadero ejemplo de superación; tu fuerza me motiva a seguir adelante cada día. A mi mejor amiga Stefy, por ser mi sostén emocional, por creer en mí con tanta firmeza incluso cuando yo misma dudo, y por tener siempre el abrazo y la palabra justa para devolverme la fé cada vez que siento perderla. Finalmente, a Sergio, mi jefe y director de tesis: gracias infinitas por tu colaboración, apoyo y ayuda para darle forma a todo este trabajo. Sin tu guía, la materialización de Aurum no habría sido posible.

Índice general

Resumen	I
1. Introducción general	1
1.1. Contexto de la banca móvil	1
1.2. Estado del arte	2
1.3. Objetivos y alcance	3
1.3.1. Alcance y limitaciones	3
1.4. Motivación: accesibilidad e inclusión	4
2. Introducción específica	7
2.1. Marco metodológico	7
2.2. Herramientas utilizadas	7
2.3. Requerimientos y modelos de IA utilizados	8
2.4. Exploración teórica de arquitecturas evaluadas	9
3. Diseño e implementación	11
3.1. Arquitectura del sistema	11
3.2. Tratamiento y preparación de datos	14
3.2.1. Creación de plantillas de comandos y aumentación de datos	14
3.2.2. Composición y distribución del corpus	14
3.2.3. Modelo de datos	15
3.3. Modelos de inteligencia artificial	15
3.3.1. Justificación de la selección algorítmica	15
3.3.2. Clasificador de intenciones: <i>Maximum Entropy</i> (MaxEnt)	16
Fundamento teórico	16
Extracción de <i>features</i> y vectorización	17
Función de decisión y umbral de confianza	17
3.3.3. Extractor de entidades: <i>Conditional Random Field</i> (CRF)	17
Fundamento teórico	17
Decodificación con el algoritmo de Viterbi	18
Implementación con <i>MLWordTagger</i>	18
3.3.4. Esquema de anotación IOB	18
3.3.5. Proceso de entrenamiento	19
Entrenamiento del clasificador de intenciones	19
Entrenamiento del extractor de entidades	19
Integración de modelos en el proyecto	20
3.3.6. Preprocesamiento: normalización Unicode	20
3.3.7. Mecanismo de inferencia de moneda	20
3.4. Desarrollo del SDK (Aurum)	21
3.4.1. Clase fachada: <i>AurumEngine</i>	21
3.4.2. Captura de audio: <i>AudioCaptureManager</i>	22
3.4.3. Reconocimiento de voz: <i>SpeechRecognizer</i>	22

3.4.4.	Normalización de texto: <code>TextNormalizer</code>	22
3.4.5.	Procesamiento NLU: <code>NLUProcessor</code>	22
3.4.6.	Modelo de datos: <code>TransactionPayload</code>	23
3.4.7.	API pública y comunicación asíncrona	23
3.4.8.	Gestión de errores: <code>AurumError</code>	24
3.5.	Desarrollo de la aplicación base	25
3.5.1.	Arquitectura MVVM y flujo de estados	25
3.5.2.	Integración con el SDK Aurum	25
3.5.3.	Capa de presentación (SwiftUI)	25
3.5.4.	Gestión de permisos y privacidad	26
3.5.5.	Flujo de interacción del usuario	26
4.	Ensayos y resultados	29
4.1.	Desempeño del modelo	29
4.1.1.	Clasificador de intenciones (MaxEnt)	29
4.1.2.	Extractor de entidades (CRF)	30
	Métricas a nivel de token	30
	Métricas a nivel de entidad	31
4.2.	Ensayo de modelos	32
4.2.1.	Evaluación de robustez lingüística	32
4.2.2.	Benchmark de latencia de inferencia	34
4.2.3.	Huella de almacenamiento del SDK	35
4.3.	Pruebas de usabilidad	36
4.3.1.	Protocolo de evaluación	36
4.3.2.	Resultados de usabilidad	39
4.4.	Análisis de resultados	43
4.4.1.	Validación de la hipótesis central	43
4.4.2.	Comparación con arquitecturas alternativas	43
4.4.3.	Rol del <code>TextNormalizer</code>	44
4.4.4.	Limitaciones del estudio	45
4.4.5.	Conexión con CRISP-DM	46
5.	Conclusiones	47
5.1.	Resultados generales	47
5.2.	Próximos pasos	48
	Bibliografía	51

Índice de figuras

3.1. Diagrama de arquitectura de componentes que conforman la prueba de concepto.	12
3.2. Flujo de pipeline de una transacción realizada por voz.	13
3.3. Diagrama de secuencia del protocolo <code>AurumDelegate</code> : comunicación asíncrona entre la aplicación base y los componentes del SDK Aurum.	24
3.4. Diagrama de flujo de Aurum.	28
4.1. Capturas de pantalla de las solicitudes de permisos en la aplicación base.	37
4.2. Pantalla principal de la aplicación base en estado idle	38
4.3. Capturas de pantalla que ilustran el funcionamiento de la transcripción en tiempo real y el procesamiento a texto.	39
4.4. Pantalla de confirmación de transferencia	40
4.5. Confirmación de consulta de saldo	41
4.6. Confirmación de intención de cancelación	42

Índice de tablas

2.1. Comparativa de arquitecturas evaluadas para NLU <i>on-device</i> en iOS. Los valores de BETO y BiLSTM corresponden a referencias bibliográficas; los de Create ML (MaxEnt/CRF) son resultados experimentales propios.	10
3.1. Distribución del corpus de clasificación de intenciones por clase y partición.	14
3.2. Distribución del corpus NER por moneda y partición.	15
3.3. Ejemplo de anotación IOB para el comando <code>pasale dos lucas a Juan Pérez</code>	19
3.4. Componentes del SDK Aurum y sus dependencias de frameworks nativos.	21
3.5. Campos de la estructura <code>TransactionPayload</code>	23
3.6. Permisos requeridos por la aplicación base y su justificación técnica.	26
4.1. Rendimiento del clasificador de intenciones (MaxEnt) evaluado sobre el conjunto de test.	29
4.2. Matriz de confusión del clasificador de intenciones sobre el conjunto de test. Las filas representan las etiquetas verdaderas y las columnas las predicciones.	30
4.3. Rendimiento del extractor de entidades (MLWordTagger, algoritmo CRF) a nivel de token individual con esquema IOB.	31
4.4. Rendimiento del extractor de entidades a nivel de entidad completa. <i>Strict</i> : coincidencia exacta de tipo y posición. <i>Relaxed</i> : coincidencia de tipo con superposición parcial de tokens.	32
4.5. Evaluación de robustez del pipeline NLU ante variaciones lingüísticas del español rioplatense y errores de STT. <i>Intent Acc.</i> : accuracy de clasificación de intención. <i>Entity F1</i> : macro F1-Score <i>strict</i> de extracción de entidades.	33
4.6. Latencia de inferencia de los modelos Core ML, medida sobre macOS (CPU). En un dispositivo iOS con Neural Engine, las latencias serían inferiores. Se reportan estadísticos sobre 1.600 inferencias por modelo.	34
4.7. Desglose del tamaño del SDK Aurum por componente.	35
4.8. Resultados de las pruebas de usabilidad por tarea. La tasa de éxito se calcula sobre el total de intentos. El tiempo de interacción incluye el habla del usuario, el procesamiento y la confirmación.	43
4.9. Resumen de rendimiento de los modelos Core ML del SDK Aurum.	46

A la memoria de mi mamá y mi abuelo. El amor y la fuerza que me dieron trascienden el tiempo y cualquier distancia.

Hoy culmino este proyecto sabiendo que, aunque no puedan sostener estas páginas en sus manos, me acompañan en el alma. Están conmigo en cada logro, y este trabajo es un testimonio de todo lo que sembraron en mí.

Capítulo 1

Introducción general

El presente capítulo enmarca el desarrollo de una solución basada en interfaces conversacionales impulsadas por inteligencia artificial. A lo largo de las siguientes secciones se analizará el estado del arte de estas tecnologías, justificando la necesidad crítica de implementar un procesamiento local de los datos para garantizar la seguridad de la información financiera, y se define de manera precisa los objetivos y el alcance metodológico de la prueba de concepto a desarrollar en el ecosistema iOS.

1.1. Contexto de la banca móvil

En la última década, la banca digital ha experimentado una transformación radical [1], al migrar de plataformas web tradicionales a aplicaciones móviles que se han consolidado como el principal punto de contacto entre los clientes y las instituciones financieras. En este entorno de alta competitividad, la innovación continua en la experiencia de usuario se ha vuelto un diferenciador estratégico clave para la retención y satisfacción del cliente. Las interfaces conversacionales, impulsadas por los recientes avances en inteligencia artificial y procesamiento de lenguaje natural, representan la siguiente frontera en la interacción humano-computadora, lo que permite una comunicación más fluida, rápida e intuitiva [2]. Este trabajo se enmarca en dicha evolución tecnológica, explorando la viabilidad de aplicar tecnologías de reconocimiento y comprensión de voz para la ejecución de operaciones financieras directamente desde el ecosistema de una aplicación de banca móvil.

En el contexto argentino en particular, la adopción de la banca móvil se aceleró de manera significativa a partir de la pandemia de COVID-19, que en 2020 forzó a las instituciones financieras a consolidar sus canales digitales como alternativa al servicio presencial [3]. Según datos del Banco Central de la República Argentina (BCRA), el número de transferencias inmediatas a través de plataformas digitales creció sostenidamente en los años subsiguientes, y el segmento de clientes que gestiona sus finanzas exclusivamente a través del canal móvil representa hoy una fracción sustancial de la base de usuarios de los principales bancos privados del país [4]. El Banco Galicia [5], como una de las entidades financieras privadas de mayor envergadura del país, no es ajeno a esta tendencia: sus plataformas digitales concentran la gran mayoría de las interacciones cotidianas de sus clientes, como consultas de saldo, transferencias, pagos de servicios y operaciones de inversión.

Es en este escenario de alta digitalización y creciente demanda de experiencias de usuario personalizadas donde la interfaz de voz emerge como una propuesta de valor diferencial. La interacción por voz reduce la carga cognitiva asociada a la navegación entre menús y formularios, y permite ejecutar operaciones en contextos donde el uso de las manos no es posible o no resulta conveniente. Esta convergencia entre la madurez tecnológica de los modelos de lenguaje y la infraestructura de hardware de los dispositivos móviles modernos crea las condiciones necesarias para desarrollar soluciones de voz robustas, seguras y completamente autónomas que no dependan de servicios externos.

1.2. Estado del arte

En la actualidad, los asistentes de voz de propósito general han alcanzado un grado de madurez notable en tareas de reconocimiento automático de habla y comprensión del lenguaje natural [6, 7]. Sin embargo, la adopción e integración de estas tecnologías en dominios de misión crítica y alta seguridad, como el sector bancario, sigue siendo limitada. Las soluciones bancarias que han incorporado interfaces conversacionales suelen basarse en integraciones en la nube, donde los flujos de audio y datos se envían a servidores externos para su procesamiento. Este enfoque tradicional genera legítimas preocupaciones respecto a la privacidad, la latencia y la exposición de información financiera sensible [8].

Entre los casos de referencia más relevantes a nivel internacional, se destaca Erica [9], el asistente virtual de Bank of America, lanzado en 2018 y considerado uno de los primeros asistentes financieros conversacionales en alcanzar escala masiva. Erica permite a los usuarios realizar consultas de saldo, revisar el historial de transacciones y recibir recomendaciones personalizadas mediante lenguaje natural. Sin embargo, su arquitectura depende del procesamiento en la nube, lo que implica la transmisión continua de datos financieros sensibles a servidores externos. De manera similar, iniciativas como Eno de Capital One [10] y los chatbots bancarios desplegados en diversas entidades europeas siguen el mismo paradigma de conectividad remota.

En el ecosistema de Apple, la integración de asistentes de voz en aplicaciones financieras ha explorado el uso de SiriKit [11], un framework que permite a los desarrolladores exponer ciertas funcionalidades de sus aplicaciones al asistente nativo del sistema operativo. No obstante, esta integración presenta limitaciones en cuanto a la personalización del flujo conversacional y a la capacidad de interpretar intenciones transaccionales complejas y específicas del dominio bancario. El control sobre el procesamiento del lenguaje natural recae sobre los servidores de Apple, lo que no garantiza el nivel de privacidad requerido para operaciones financieras sensibles.

Para mitigar estos riesgos, la tendencia tecnológica emergente apunta hacia el procesamiento en el dispositivo. Bajo este paradigma, los modelos de inteligencia artificial se ejecutan localmente utilizando el hardware del dispositivo móvil, lo que garantiza que los datos acústicos y las intenciones transaccionales del usuario nunca abandonen su teléfono. Esta arquitectura maximiza la seguridad y la confidencialidad, pilares fundamentales para la viabilidad de cualquier solución dentro de la industria financiera. El avance en la comprensión de modelos de lenguaje, técnicas como la cuantización y la destilación de conocimiento, han hecho factible ejecutar modelos de comprensión del lenguaje natural de alta precisión

directamente sobre el procesador neuronal (*Neural Engine*) de los chips de la serie A y M de Apple [12, 13], lo que abrió una nueva generación de aplicaciones de IA privadas y sin latencia de red.

1.3. Objetivos y alcance

El objetivo principal de este trabajo es diseñar, desarrollar y evaluar un prototipo funcional de un asistente transaccional por voz integrado en el entorno de una aplicación móvil para iOS. La solución se materializa en un SDK (*Software Development Kit*) nativo denominado Aurum, que encapsula los modelos de *machine learning* y la lógica de procesamiento necesaria para interpretar instrucciones financieras dictadas por voz. Este prototipo servirá como una prueba de concepto destinada a validar la viabilidad técnica de un sistema que permita a los usuarios ejecutar operaciones financieras mediante comandos de voz en lenguaje natural. Si bien el presente trabajo se desarrolla y valida en el contexto del Banco Galicia, la arquitectura de Aurum fue concebida como un componente reutilizable e independiente de la entidad financiera: cualquier aplicación bancaria para iOS podría integrar el SDK implementando su protocolo público de comunicación, sin necesidad de modificar la lógica interna del *framework*.

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

- Mejorar la accesibilidad e inclusión: desarrollar un canal de interacción alternativo que empodere a personas con discapacidades visuales o motoras, que reduzca la dependencia de la navegación táctil tradicional.
- Garantizar la seguridad y privacidad de los datos: implementar modelos de machine learning optimizados para ejecutarse de manera estrictamente local mediante Core ML de Apple [14], asegurando que el audio y los datos sensibles del usuario se procesen íntegramente en el dispositivo.
- Optimizar la experiencia de usuario: simplificar operaciones bancarias complejas condensándolas en un único paso conversacional, y reduzca de esta manera la fricción y el tiempo necesario para completar una transacción.
- Sentar bases técnicas para futuras auditorías: generar la documentación arquitectónica y técnica necesaria para facilitar una futura evaluación y validación por parte del equipo de ciberseguridad de la entidad bancaria.

1.3.1. Alcance y limitaciones

Con el propósito de mantener el enfoque en la validación tecnológica del motor de inteligencia artificial y la experiencia de usuario, el trabajo se delimita bajo parámetros estrictos de alcance.

- Dentro del alcance: el desarrollo contempla la creación del SDK Aurum como *framework* dinámico reutilizable y una aplicación prototipo nativa para iOS que lo consume, ambos operando de manera independiente y aislada. El núcleo técnico abarcará la implementación del *pipeline* de inteligencia artificial en el dispositivo físico, integrando la captura segura de audio, un modelo de reconocimiento automático de habla y un módulo de interpretación de lenguaje natural para extraer intenciones y entidades clave. La aplicación base actúa como cliente de validación del SDK, lo que demuestra

que la integración se reduce a implementar un único protocolo de delegación. Dado el carácter de la prueba de concepto, todo el flujo de transacción, la pantalla de confirmación y los resultados de las operaciones se simularán utilizando datos completamente ficticios.

- Fuera del alcance: el trabajo no contempla la integración del prototipo, ni de sus componentes, con el código fuente de la aplicación productiva oficial, ni la conexión con APIs, servicios de backend o bases de datos reales de la institución financiera. Asimismo, queda excluida la puesta en marcha en producción y la publicación en la App Store. Desde el punto de vista del manejo de la información, está estrictamente excluido el uso o acceso a datos reales de clientes, así como la ejecución de transacciones que movi-licen dinero real. Finalmente, el sistema se limitará al español rioplatense focalizado, dejando fuera del alcance el soporte multilingüe y operaciones financieras distintas a transferencias y consultas.

1.4. Motivación: accesibilidad e inclusión

El paradigma de diseño predominante en las aplicaciones de banca móvil actuales se basa casi exclusivamente en la interacción táctil y visual. Si bien este enfoque resulta eficiente para la mayoría de los usuarios, presenta barreras tecnológicas y de usabilidad significativas para grupos demográficos importantes. Por un lado, las personas con discapacidades o disminuciones visuales enfrentan el desafío de navegar a través de menús complejos, campos de texto y botones, tareas que a menudo resultan frustrantes incluso con el soporte de las herramientas de accesibilidad nativas de los sistemas operativos. Por otro lado, las personas con discapacidades motoras encuentran que la necesidad de una manipulación precisa de la pantalla dificulta o impide directamente el uso autónomo de la aplicación para realizar operaciones financieras cotidianas.

En Argentina, según datos del Instituto Nacional de Estadística y Censos (INDEC), aproximadamente el 10 % de la población presenta algún tipo de dificultad o limitación permanente [15]. Dentro de este universo, las limitaciones visuales y motoras representan una proporción significativa, lo que configura un segmento de usuarios que históricamente ha sido desatendido por el diseño centrado en la interfaz táctil. A nivel global, la Organización Mundial de la Salud (OMS) estima que más de 2.200 millones de personas presentan alguna forma de deficiencia visual, de las cuales una fracción importante se encuentra en edad económicamente activa y requiere acceso a servicios financieros [16]. La brecha entre la necesidad de acceso a servicios bancarios y la capacidad de las interfaces actuales para satisfacer dicha necesidad constituye un problema de equidad e inclusión que la tecnología tiene el potencial y la responsabilidad de resolver.

La motivación principal de este trabajo es derribar estas barreras tecnológicas, promoviendo una inclusión financiera real mediante el desarrollo de un canal de interacción alternativo. La implementación de un asistente transaccional por voz no solo genera un impacto social positivo al mejorar radicalmente la accesibilidad, sino que también ofrece una experiencia de usuario superior, simplificando operaciones complejas en un único paso conversacional y reduzca la fricción del proceso. Más aún, al garantizar que todo el procesamiento ocurra de forma local en el dispositivo, se elimina la resistencia que muchos usuarios podrían tener

hacia el uso de comandos de voz en entornos de alta sensibilidad, como lo es la gestión de sus finanzas personales.

Capítulo 2

Introducción específica

El desarrollo de una solución basada en el procesamiento de lenguaje natural exige un acoplamiento profundo y eficiente con el hardware del dispositivo móvil. Este apartado detalla el ecosistema tecnológico y de software seleccionado para el desarrollo del prototipo.

2.1. Marco metodológico

Para la concepción, desarrollo y evaluación del sistema de transacciones financieras por voz, se adoptó como marco de trabajo la metodología CRISP-DM (*Cross-Industry Standard Process for Data Mining*) [17]. Si bien CRISP-DM fue concebida originalmente para proyectos de minería de datos, su estructura iterativa y centrada en el ciclo de vida de los datos la convierte en el estándar de facto para la ingeniería de proyectos de inteligencia artificial y machine learning. La elección de esta metodología responde a la necesidad de contar con un enfoque estructurado que permita alinear los objetivos técnicos del procesamiento de lenguaje natural con los requisitos críticos y las restricciones de seguridad propios de una aplicación de banca móvil. El ciclo de vida se ha adaptado específicamente para abordar el desarrollo de un pipeline NLU (*Natural Language Understanding*) [18] de ejecución local dentro del ecosistema iOS.

2.2. Herramientas utilizadas

Con el objetivo de cumplir con los requerimientos de procesamiento local (*Edge AI*) [19] y garantizar la privacidad por diseño (*Privacy by Design*) [20] en el contexto de una aplicación financiera, se adoptó de manera integral el entorno nativo de Apple. A continuación, se enumeran las herramientas integradas en la arquitectura del sistema.

- Swift [21]: es un lenguaje de programación compilado, multiparadigma y de tipado estático fuerte, desarrollado por Apple para la creación de software en sus sistemas operativos. Su adopción se justifica por su alto rendimiento computacional y su seguridad en el manejo de memoria mediante el Conteo Automático de Referencias (ARC). Estas características son esenciales para manipular flujos de datos en tiempo real (como el audio) y procesar estructuras de datos complejas sin incurrir en fugas de memoria o vulnerabilidades durante la ejecución transaccional.
- Xcode [22]: es el Entorno de Desarrollo Integrado (IDE) oficial y nativo provisto por Apple, que incluye los compiladores, depuradores y simuladores

necesarios para el ciclo de vida del software. Se utilizó como la herramienta central para la codificación, compilación y empaquetado del SDK. Su uso es mandatorio para desplegar la prueba de concepto en dispositivos físicos y para utilizar la herramienta *Instruments*, esta permitió perfilar y auditar el consumo de memoria y CPU de los modelos de Inteligencia Artificial durante la inferencia local.

- AVFoundation [23]: es el *framework* nativo de Apple que proporciona una interfaz de programación de bajo nivel para interactuar con el hardware audiovisual del dispositivo. Su uso fue indispensable para orquestar la captura segura del comando de voz. Específicamente, se utilizó la clase `AVAudioSession` para gestionar los permisos de privacidad del micrófono a nivel del sistema operativo, y la clase `AVAudioEngine` para interceptar la onda sonora cruda y aislar el flujo acústico sin depender de librerías de terceros.
- Speech Framework [24]: es una Interfaz de Programación de Aplicaciones (API) de Apple dedicada exclusivamente al Reconocimiento Automático de Habla (ASR), capaz de transcribir señales de audio a cadenas de texto. Se justificó su uso frente a otras alternativas comerciales debido a su capacidad de operar en modo *offline*. Al forzar la propiedad `requiresOnDevice-Recognition`, esta herramienta garantizó que la voz del usuario nunca fuera transmitida a servidores externos, mitigando los vectores de ataque y cumpliendo con el estándar de secreto bancario requerido por el proyecto.
- Core ML [25]: es el *framework* central de *Machine Learning* de Apple, diseñado para integrar, optimizar y ejecutar modelos predictivos directamente en el sistema operativo iOS. Constituye el motor analítico del proyecto. Se seleccionó porque delega automáticamente las cargas matemáticas pesadas (como la clasificación de intenciones y la extracción de entidades) al *Apple Neural Engine* (ANE). Esto garantiza que los algoritmos de Procesamiento de Lenguaje Natural (PLN) se ejecuten con una latencia mínima y un impacto energético marginal, habilitando el paradigma de inferencia 100 % *On-Device*.
- Coremltools [26]: es un paquete de código abierto basado en Python, desarrollado por Apple, que unifica la conversión de modelos entrenados en librerías estándar (como PyTorch o TensorFlow) al formato propietario `.mlpackage`. Su aplicación metodológica fue crítica para viabilizar el despliegue de las redes neuronales en el entorno móvil. Se utilizó para aplicar técnicas de cuantización a los pesos sinápticos de los modelos, reduciendo drásticamente su tamaño de almacenamiento y la huella en la memoria RAM del dispositivo, sin comprometer significativamente la exactitud predictiva del sistema.

2.3. Requerimientos y modelos de IA utilizados

El núcleo algorítmico del prototipo aborda la comprensión del comando de voz como un problema predictivo dual de machine learning: la clasificación de intenciones y el etiquetado de entidades transaccionales.

Por un lado, la clasificación multiclase a nivel de oración tiene como objetivo inferir la acción principal del usuario (por ejemplo, iniciar una transferencia o consultar un saldo). Matemáticamente, la red neuronal proyecta la secuencia de texto vectorizada hacia un espacio discreto de clases predefinidas, empleando una función de activación *softmax* para seleccionar, de manera excluyente, la intención que maximiza la probabilidad estadística.

Por otro lado, la conformación del *payload* financiero exige resolver un problema de etiquetado de secuencias a nivel de *token*, enmarcado en el Reconocimiento de Entidades Nombradas (NER)[27]. En lugar de emitir un único juicio para toda la oración, el modelo asigna una clase categórica a cada palabra utilizando el esquema de anotación IOB (*Inside*, *Outside*, *Beginning*). Esta técnica permite al sistema demarcar los límites exactos de variables críticas, como el monto numérico, la divisa y el destinatario, capturando las dependencias semánticas del lenguaje para extraer los parámetros exactos de la operación.

2.4. Exploración teórica de arquitecturas evaluadas

Para abordar este desafío predictivo dual y que se respeten las restricciones de memoria y procesamiento de los dispositivos móviles, se llevó a cabo una evaluación teórica de diversas arquitecturas algorítmicas, que analiza el compromiso (*trade-off*) entre precisión semántica, latencia de inferencia y consumo de recursos.

En primer lugar, se analizó el estado del arte representado por las arquitecturas basadas en Transformers. Este paradigma, fundamentado en el mecanismo de autoatención, prescinde de la recurrencia secuencial y permite capturar dependencias a largo plazo y procesa el contexto de manera bidireccional y paralelizable. Dentro de esta familia, se evaluó la viabilidad de incorporar modelos del lenguaje preentrenados adaptados al español, como BETO (la variante hispanohablante de BERT) [28]. Si bien BETO ofrece un rendimiento superior en la comprensión de las sutilezas léxicas y morfológicas del lenguaje coloquial rioplatense, su implementación nativa en el edge presenta obstáculos significativos. La alta dimensionalidad de sus parámetros (típicamente superior a 110 millones) exige la aplicación de técnicas agresivas de compresión, poda (*pruning*) y destilación de conocimiento (*knowledge distillation*) para transformarlo en un artefacto viable para el entorno iOS sin agotar la memoria RAM del dispositivo.

Como alternativa de menor huella computacional, se exploró la familia de las redes neuronales recurrentes (RNN) [29]. A diferencia de los transformers, las RNN procesan los tokens de manera estrictamente secuencial, actualiza un estado oculto que actúa como memoria temporal del contexto lingüístico. Dado que las arquitecturas RNN tradicionales son susceptibles al problema del desvanecimiento del gradiente (*vanishing gradient*) al procesar secuencias largas, la investigación se centró en sus variantes con mecanismos de compuerta (*gating*): las redes de memoria a corto y largo plazo (LSTM) [30] y las unidades recurrentes con compuerta (GRU) [31]. Estas topologías regulan matemáticamente el flujo de información, dice qué características del contexto previo deben retenerse y cuáles descartarse. Históricamente, las arquitecturas basadas en LSTM bidireccionales (BiLSTM) [32] han demostrado una alta eficacia para el etiquetado de secuencias (NER) [27], logra una precisión competitiva con un costo computacional y tamaño de modelo órdenes de magnitud inferiores a los de un modelo fundacional, lo que las hace inherentemente más compatibles con inferencias On-Device [33].

Finalmente, se evaluó la pertinencia de utilizar los frameworks de comprensión de lenguaje natural nativos de Apple, en particular la API NaturalLanguage y el ecosistema de Create ML [34]. Apple proporciona clases altamente especializadas, como `NLModel` y `NLTagger`, diseñadas para compilar clasificadores de texto y extractores de entidades a partir de *datasets* etiquetados. La ventaja superlativa de este enfoque reside en su integración opaca y de bajo nivel con el silicio del dispositivo. Los modelos generados bajo este estándar están nativamente optimizados para aprovechar la aceleración por hardware del procesamiento neural de Apple, garantiza tiempos de latencia del orden de los milisegundos y un impacto insignificante en la autonomía de la batería. Aunque la utilización de frameworks cerrados sacrifica el grado de control arquitectónico y la hiperparametrización profunda que ofrecen librerías como PyTorch o TensorFlow, su robustez operativa, la garantía de privacidad al aislar el procesamiento de la red, y la facilidad de integración mediante Core ML, posicionan a estas herramientas nativas como candidatos estratégicos para validar la viabilidad técnica de una prueba de concepto en el sector financiero.

A continuación se detallan las métricas que respaldan lo mencionado en el presente capítulo:

TABLA 2.1. Comparativa de arquitecturas evaluadas para NLU *on-device* en iOS. Los valores de BETO y BiLSTM corresponden a referencias bibliográficas; los de Create ML (MaxEnt/CRF) son resultados experimentales propios.

Criterio	BETO (Transformer)	BiLSTM (RNN)	Create ML (MaxEnt)	Create ML (CRF)
Tamaño del modelo (MB)	~440	~5–15	<1	<1
Parámetros (millones)	~110	~0.5–2	N/A	N/A
Latencia de inferencia (ms)	>200	20–50	<10	<10
Memoria RAM en runtime (MB)	>300	15–40	<5	<5
Requiere conversión Core ML	Sí	Sí	No	No
Aceleración Neural Engine	Parcial	Parcial	Nativa	Nativa
Entrenamiento sin GPU	No viable	Posible	Sí (CPU)	Sí (CPU)
Privacidad <i>on-device</i>	Sí	Sí	Sí	Sí

Capítulo 3

Diseño e implementación

Este capítulo expone el proceso técnico de implementación y desarrollo del sistema propuesto. En primer lugar, se describe la arquitectura de la solución, en la que se definió la separación de responsabilidades entre la aplicación base y el SDK nativo denominado Aurum. A continuación, se detalla el modelo de datos diseñado para encapsular las intenciones y entidades extraídas de los comandos de voz. Finalmente, se explican los componentes del código fuente del prototipo.

3.1. Arquitectura del sistema

El diseño arquitectónico se fundamenta en el paradigma de computación *Edge AI*, que traslada la carga computacional de los algoritmos de inteligencia artificial directamente al dispositivo móvil del usuario. Esta decisión persigue un doble objetivo: minimizar la latencia inherente a las comunicaciones de red y garantizar un entorno de privacidad por diseño. Al ejecutar los modelos a través del *framework* Core ML, los flujos de audio y las transcripciones de índole financiera no abandonan en ningún momento el hardware de Apple.

El sistema fue concebido bajo el principio de separación de responsabilidades y se estructura en dos bloques funcionales independientes: la aplicación base anfitriona y el SDK nativo Aurum [35]. Es necesario establecer una restricción arquitectónica crítica: el proyecto no incluye algoritmos de validación biométrica de voz. La autenticación del cliente, el manejo de tokens de sesión y las medidas de seguridad perimetral recaen exclusivamente sobre la aplicación móvil del banco. El SDK Aurum opera bajo la premisa de que es invocado dentro de un contexto transaccional previamente asegurado por la entidad financiera, y limita su dominio de responsabilidad a la captura acústica, la transcripción y la extracción semántica de intenciones y entidades.

El proceso de desarrollo se articuló siguiendo las fases de la metodología CRISP-DM:

- **Comprensión del negocio:** el problema central radica en las barreras de usabilidad que las interfaces táctiles imponen a personas con discapacidades visuales o motoras. La solución propuesta habilita un canal conversacional que transforma lenguaje natural, inherentemente ambiguo, en formatos estructurados que el *core* bancario pueda procesar. Esto implica capturar una emisión acústica y extraer inequívocamente una intención principal (clasificación de intención) y los parámetros asociados (reconocimiento de entidades nombradas: monto, moneda y destinatario).

- **Comprensión de los datos:** ante la ausencia de registros históricos de comandos de voz y las restricciones de privacidad del sector bancario, se determinó la necesidad de construir un *dataset* sintético. Se analizó la morfología del lenguaje coloquial rioplatense aplicado a finanzas, identificando sinónimos de acciones, variaciones en la expresión de montos y formas comunes de nombrar contactos.
- **Evaluación:** se midió el desempeño predictivo mediante un conjunto de prueba aislado, con métricas de *Accuracy* para la clasificación de intenciones y *Precision*, *Recall* y *F1-Score* para el reconocimiento de entidades. A nivel de integración, se realizaron pruebas de usabilidad en el entorno simulado de iOS, midiendo la latencia de inferencia local y la tasa de éxito de la tarea integral.
- **Despliegue:** el despliegue consistió en la conversión de los modelos al formato *.mlpackage*, su integración directa en el proyecto Xcode y su invocación local mediante Core ML. El modelo se desplegó encapsulado dentro del propio binario de la aplicación iOS, garantizando que el *pipeline* completo se ejecute de manera estrictamente local.

En la figura 3.1 se presenta el diagrama de componentes.

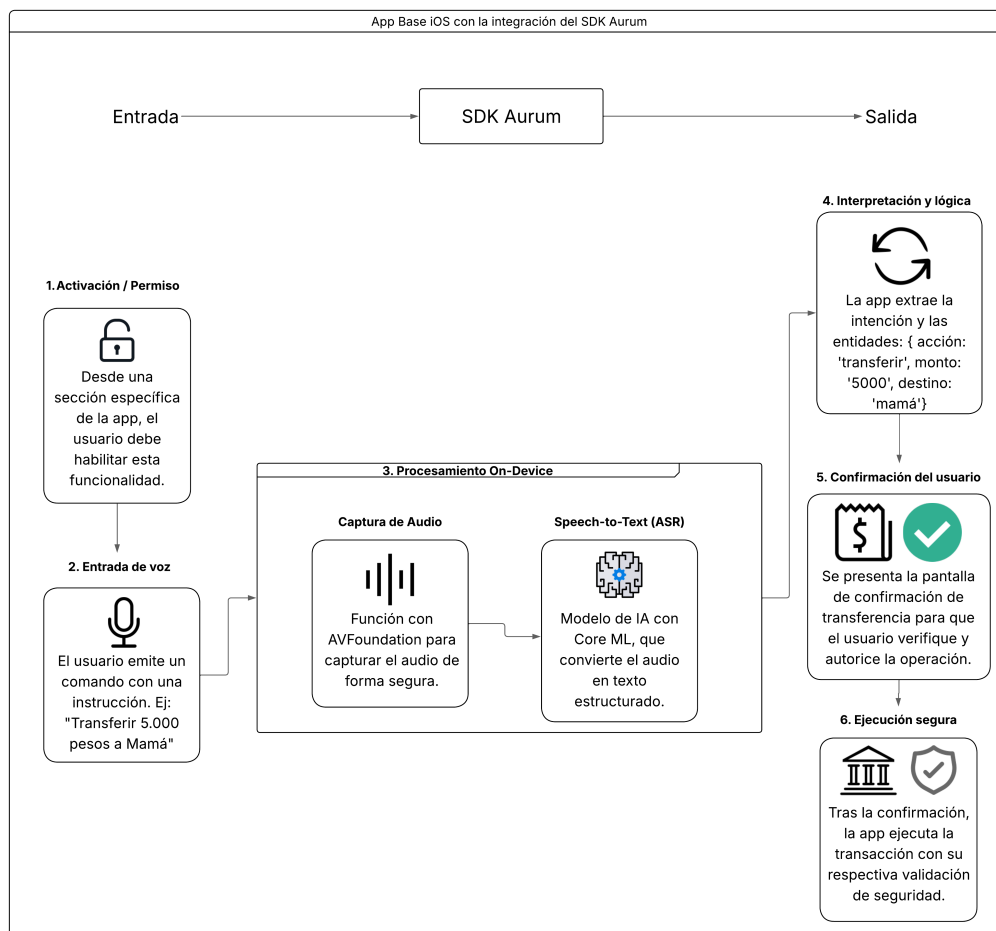


FIGURA 3.1. Diagrama de arquitectura de componentes que conforman la prueba de concepto.

El ciclo de vida de una transacción por voz atraviesa un *pipeline* determinista. El proceso se desencadena cuando el usuario interactúa con el botón de dictado de la interfaz gráfica, momento en que la aplicación invoca la API pública del SDK. El SDK inicializa la captura de audio mediante AVFoundation, ejecuta la transcripción fuera de línea con el *Speech Framework*, normaliza el texto resultante y lo inyecta en el motor de comprensión del lenguaje natural. Los modelos Core ML resuelven la clasificación de intención y la extracción de entidades. Finalmente, la salida se traduce en un *payload* estructurado y fuertemente tipado, que se retorna de manera asíncrona a la aplicación anfitriona a través del patrón de delegación [21]. A partir de ese momento, el SDK cede el control a la aplicación base, que asume la responsabilidad de presentar la interfaz de confirmación y comunicarse con el *core* bancario.

En la figura 3.2 se muestra el flujo del *pipeline*.

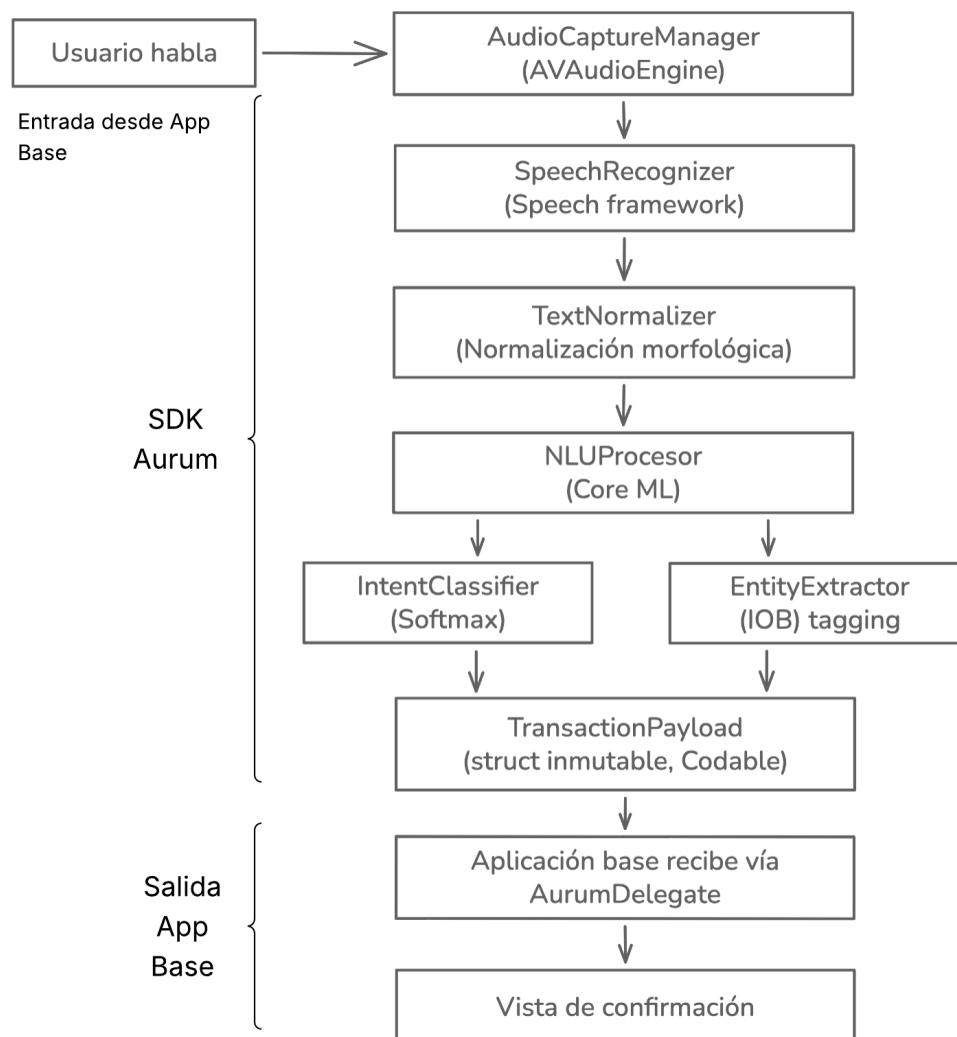


FIGURA 3.2. Flujo de pipeline de una transacción realizada por voz.

3.2. Tratamiento y preparación de datos

El desarrollo de modelos de procesamiento de lenguaje natural aplicados al sector bancario enfrenta el problema del arranque en frío (*cold start*): las normativas de privacidad y el secreto bancario imposibilitan el uso de corpus conversacionales de clientes reales, y al tratarse de una prueba de concepto no existe telemetría histórica de comandos transaccionales. Ante esta restricción, se determinó la generación de un conjunto de datos sintéticos (*mock data*), lo que permite superar la ausencia de datos iniciales y ejercer control sobre la distribución probabilística de las clases y la cobertura de los casos límite.

3.2.1. Creación de plantillas de comandos y aumentación de datos

La síntesis del corpus se estructuró a partir de plantillas gramaticales que modelan las posibles estructuras sintácticas de interacción con el sistema bancario, abstrayendo los valores concretos: [ACCIÓN] [MONTO] [MONEDA] a [DESTINATARIO]. Una vez consolidadas las estructuras base, se implementaron técnicas de aumentación de datos (*data augmentation*) para inyectar la variabilidad inherente al lenguaje natural [36].

Los datos resultantes de la inferencia se estructuraron utilizando *structs* en Swift [21], garantizando la inmutabilidad de la información una vez procesada. Para la serialización se adoptó el protocolo *Codable* nativo de Apple [37], estandarizando la comunicación entre el SDK y la aplicación base mediante formatos universales como JSON [38].

3.2.2. Composición y distribución del corpus

El proceso de aumentación produjo dos corpus diferenciados, uno para cada tarea de *machine learning*: clasificación de intenciones y extracción de entidades (NER). Ambos se partitionaron en una proporción 80/20 entre entrenamiento y test, respectivamente.

La tabla 3.1 detalla la distribución del corpus de intenciones por clase.

TABLA 3.1. Distribución del corpus de clasificación de intenciones por clase y partición.

Clase	Train	Test	Total
transferencia	1275 (34,6 %)	325 (35,3 %)	1600
consulta_saldo	1195 (32,5 %)	305 (33,2 %)	1500
cancelar	1210 (32,9 %)	290 (31,5 %)	1500
Total	3680	920	4600

La distribución entre clases se mantuvo aproximadamente uniforme para evitar sesgos en el clasificador. La clase TRANSFERENCIA concentra una proporción ligeramente mayor debido a la incorporación de muestras con variaciones coloquiales de moneda, necesarias para cubrir la diversidad léxica del español rioplatense.

Para el corpus de extracción de entidades, cada muestra corresponde a un comando de transferencia tokenizado con etiquetas IOB (*Inside-Outside-Beginning*).

La tabla 3.2 presenta la distribución por moneda, dado que el balance entre pesos argentinos (ARS) y dólares estadounidenses (USD) resulta crítico para que el modelo reconozca ambas divisas con precisión comparable.

TABLA 3.2. Distribución del corpus NER por moneda y partición.

Moneda	Train	Test	Total
ARS (pesos, mangos, lucas)	1335 (52,4 %)	355 (55,5 %)	1690
USD (dólares, verdes)	1215 (47,6 %)	285 (44,5 %)	1500
Total	2550	640	3190

Las técnicas de aumentación aplicadas incluyeron la inyección de variaciones propias del español rioplatense: formas verbales en voseo (*transferile*, *mandáale*, *pasale*), expresiones coloquiales de moneda (*lucas* como multiplicador de miles en pesos, *verdes* como sinónimo de dólares, *mangos* como sinónimo de pesos) y variaciones en la estructura sintáctica del comando (con y sin la preposición *en* antes de la moneda, con y sin el prefijo *plata* tras el verbo). Del total de muestras de transferencia, el 76 % incorpora alguna forma de voseo rioplatense, lo que refleja el registro lingüístico predominante en la población objetivo del sistema.

3.2.3. Modelo de datos

El *payload* que los modelos devuelven al sistema se consolidó en una única entidad semántica con campos fuertemente tipados: Intención (acción directiva inferida, como iniciar una transferencia), Monto (valor numérico de tipo *Double*), Moneda (enumeración con valores finitos como ARS o USD, que elimina la ambigüedad de sinónimos coloquiales) y Destinatario (cadena de texto con la identidad nominal del beneficiario). Este enfoque de modelado determinista abstrae a la aplicación anfitriona de toda la complejidad inherente al procesamiento de lenguaje natural, permitiéndole consumir un *payload* limpio cuya estructura es análoga a la que generaría una interfaz táctil tradicional basada en formularios.

3.3. Modelos de inteligencia artificial

Tras la exploración teórica de arquitecturas presentada en el capítulo 2, se seleccionaron los algoritmos provistos de forma nativa por el ecosistema de Apple a través de Create ML: un clasificador de *Maximum Entropy* (MaxEnt) para la clasificación de intenciones y un *Conditional Random Field* (CRF) para la extracción de entidades nombradas. Esta sección detalla los fundamentos algorítmicos de cada modelo, el esquema de anotación adoptado, el proceso de entrenamiento y las decisiones de preprocesamiento que determinan la calidad de la inferencia.

3.3.1. Justificación de la selección algorítmica

La decisión de adoptar los algoritmos nativos de Create ML responde a la convergencia de tres restricciones del proyecto: ejecución íntegramente local (*on-device*), tamaño de modelo compatible con el almacenamiento de un dispositivo móvil, y

latencia de inferencia imperceptible para el usuario. Como se analizó en la comparativa de arquitecturas del capítulo 2 (tabla 2.1), tanto las arquitecturas basadas en Transformers (BETO) como las redes recurrentes (BiLSTM) ofrecen una capacidad de modelado superior, pero imponen requisitos de memoria y cómputo que dificultan su despliegue directo en el *edge* sin recurrir a técnicas de compresión.

Los modelos de Create ML, en cambio, están diseñados nativamente para el ecosistema de Apple. Al compilarse a formato `.mlpackage`, delegan la ejecución al *Neural Engine* del dispositivo, lo que garantiza latencias del orden de fracciones de milisegundo y un consumo energético marginal. Adicionalmente, el entrenamiento con Create ML no requiere GPU dedicada y puede ejecutarse íntegramente en la CPU de un equipo macOS, lo que reduce la barrera de entrada para la iteración rápida sobre el corpus.

El compromiso inherente a esta elección es la menor expresividad representacional frente a modelos de *deep learning*: MaxEnt y CRF operan sobre *features* manualmente definidas (o extraídas automáticamente por Create ML a partir de *n-grams*), sin la capacidad de aprender representaciones contextuales profundas como las que generan los mecanismos de autoatención de los Transformers. No obstante, para el dominio acotado de transacciones bancarias, con un vocabulario finito y estructuras sintácticas predecibles, esta limitación resulta aceptable, como lo confirman los resultados experimentales presentados en el capítulo 4.

3.3.2. Clasificador de intenciones: *Maximum Entropy* (MaxEnt)

El clasificador de intenciones resuelve un problema de clasificación multiclase a nivel de oración: dado un comando de voz transcrito y normalizado, el modelo debe asignarle una de las tres intenciones soportadas (TRANSFERENCIA, CONSULTA_SALDO, CANCELAR).

Fundamento teórico

El modelo de *Maximum Entropy* [39], también conocido como regresión logística multinomial, se fundamenta en el principio de máxima entropía de Jaynes: entre todas las distribuciones de probabilidad consistentes con las restricciones observadas en los datos, se selecciona aquella que maximiza la entropía, es decir, la que realiza el menor número de suposiciones adicionales. Formalmente, la probabilidad de asignar la clase c a una entrada x se define como 3.1:

$$P(c | x) = \frac{1}{Z(x)} \exp \left(\sum_i w_i \cdot f_i(x, c) \right) \quad (3.1)$$

donde $f_i(x, c)$ son funciones de *features* binarias o reales que capturan propiedades relevantes de la entrada (por ejemplo, la presencia de un *n-gram* específico), w_i son los pesos aprendidos durante el entrenamiento, y $Z(x)$ es la función de partición que normaliza la distribución para que las probabilidades sumen 1 3.2:

$$Z(x) = \sum_{c' \in C} \exp \left(\sum_i w_i \cdot f_i(x, c') \right) \quad (3.2)$$

El entrenamiento consiste en encontrar los pesos w_i que maximizan la log-verosimilitud condicional sobre el corpus etiquetado, típicamente mediante optimización por gradiente (L-BFGS o gradiente descendente estocástico) con regularización L1 o L2 para prevenir el sobreajuste.

Extracción de *features* y vectorización

Create ML implementa la extracción de *features* de forma automática a través de `MLTextClassifier`. El proceso de vectorización transforma cada comando textual en un vector numérico de alta dimensión mediante la combinación de unigramas y bigramas de palabras. Para un comando como `pasale dos lucas a Pedro`, el extractor genera *features* del tipo: presencia de `pasale`, presencia de `lucas`, presencia del bigrama `dos lucas`, presencia del bigrama `a Pedro`, entre otras. Este conjunto de *features* dispersas (*sparse*) alimenta el modelo MaxEnt, que aprende qué combinaciones son discriminativas para cada intención.

La ventaja de este enfoque sobre una representación de *bag-of-words* puro es que los bigramas capturan colocaciones relevantes: `cuánto tengo` es un indicador fuerte de `CONSULTA_SALDO`, mientras que `a Pedro` lo es de `TRANSFERENCIA`. La desventaja es que no modela dependencias de largo alcance, lo cual resulta aceptable dado que los comandos bancarios raramente exceden las 15 palabras.

Función de decisión y umbral de confianza

Durante la inferencia, el modelo calcula las probabilidades *softmax* para cada clase y selecciona la de mayor probabilidad. Sin embargo, en una aplicación financiera, aceptar ciegamente la clase más probable puede conducir a la ejecución de operaciones incorrectas cuando el comando es ambiguo. Por esta razón, se implementó un umbral de confianza configurable de 0,65: si la probabilidad máxima no lo alcanza, el sistema asigna la intención `.desconocida` y solicita al usuario que reformule el comando. Este umbral fue calibrado empíricamente, buscando un punto de equilibrio entre la tasa de rechazo de comandos legítimos (*false negatives*) y la aceptación errónea de comandos ambiguos (*false positives*).

3.3.3. Extractor de entidades: *Conditional Random Field* (CRF)

La extracción de entidades nombradas (NER) constituye un problema de etiquetado de secuencias: dado un comando tokenizado, el modelo debe asignar a cada token una etiqueta que identifique si pertenece a una entidad transaccional (monto, moneda, destinatario) o no.

Fundamento teórico

A diferencia de los clasificadores independientes que etiquetan cada token de forma aislada, los *Conditional Random Fields* [40] modelan la distribución condicional de la secuencia completa de etiquetas $\mathbf{y} = (y_1, y_2, \dots, y_T)$ dado el texto de entrada $\mathbf{x} = (x_1, x_2, \dots, x_T)$ como figura en la ecuación 3.3:

$$P(\mathbf{y} \mid \mathbf{x}) = \frac{1}{Z(\mathbf{x})} \exp \left(\sum_{t=1}^T \sum_k \lambda_k \cdot f_k(y_{t-1}, y_t, \mathbf{x}, t) \right) \quad (3.3)$$

donde f_k son funciones de *features* que dependen de la etiqueta actual y_t , la etiqueta anterior y_{t-1} (modelando transiciones), la secuencia de entrada x y la posición t ; λ_k son los pesos aprendidos; y $Z(x)$ es la función de partición que normaliza sobre todas las secuencias de etiquetas posibles.

La ventaja fundamental del CRF sobre un clasificador *token-by-token* es que modela las dependencias entre etiquetas consecutivas. Esto es crucial para el esquema IOB: una etiqueta I-MONTO solo es válida si la etiqueta anterior es B-MONTO o I-MONTO. El CRF aprende estas restricciones de transición directamente de los datos, lo que reduce drásticamente la producción de secuencias inválidas como I-MONTO después de B-DESTINATARIO.

Decodificación con el algoritmo de Viterbi

Durante la inferencia, encontrar la secuencia de etiquetas más probable requiere evaluar $|C|^T$ combinaciones posibles (donde $|C|$ es el número de etiquetas y T la longitud de la secuencia), lo cual es computacionalmente intratable. El algoritmo de Viterbi [41] resuelve este problema mediante programación dinámica, encontrando la secuencia óptima en tiempo $O(T \cdot |C|^2)$. Para el caso del SDK Aurum, con 7 etiquetas IOB y comandos de hasta 15 tokens, el espacio de búsqueda se reduce de $7^{15} \approx 4,7 \times 10^{12}$ combinaciones a $15 \times 49 = 735$ operaciones, lo que explica la latencia submilisecondo de la inferencia.

Implementación con MLWordTagger

Create ML expone el entrenamiento de modelos CRF a través de la clase `MLWordTagger`, que acepta corpus en formato JSON con pares de tokens y etiquetas. La API abstrae la complejidad del entrenamiento (extracción de *features*, optimización de pesos, regularización) y genera un modelo `.mlmodel` listo para integración con Core ML. Internamente, `MLWordTagger` extrae *features* a nivel de token que incluyen: el token actual y sus vecinos (ventana contextual), prefijos y sufijos de longitud variable, la presencia de dígitos o caracteres especiales, y la capitalización. Estas *features* alimentan el CRF lineal, que aprende simultáneamente los pesos de emisión (qué *features* son indicativas de cada etiqueta) y los pesos de transición (qué pares de etiquetas consecutivas son válidos).

3.3.4. Esquema de anotación IOB

El esquema IOB (*Inside-Outside-Beginning*) [42] es el estándar de facto para la anotación de entidades nombradas en tareas de etiquetado de secuencias. Cada token recibe una de tres categorías: B-X (*Beginning*) marca el primer token de una entidad de tipo X, I-X (*Inside*) marca los tokens subsiguientes de la misma entidad, y O (*Outside*) marca los tokens que no pertenecen a ninguna entidad.

Para el dominio del SDK Aurum, se definieron tres tipos de entidades, lo que resulta en siete etiquetas posibles: B-MONTO, I-MONTO, B-MONEDA, I-MONEDA, B-DESTINATARIO, I-DESTINATARIO y O. La tabla 3.3 ilustra la anotación de un comando típico.

TABLA 3.3. Ejemplo de anotación IOB para el comando `pasale dos lucas a Juan Pérez`.

Token	pasale	dos	lucas	a	Juan	Pérez
Etiqueta	O	B-MONTO	B-MONEDA	O	B-DESTINATARIO	I-DESTINATARIO

La elección de IOB frente a esquemas alternativos como IOB2, IOBES o BILOU responde a la compatibilidad nativa con `MLWordTagger` de Create ML, que utiliza IOB como formato de anotación estándar. Para entidades de un solo token (como `lucas` en el ejemplo), la etiqueta `B-X` es suficiente. Para entidades multi-token (como `Juan Pérez`), la combinación `B-X + I-X` demarca los límites con precisión.

La reconstrucción de entidades a partir de las etiquetas IOB sigue un algoritmo lineal de complejidad $O(T)$: se recorre la secuencia de etiquetas de izquierda a derecha; al encontrar `B-X`, se inicia una nueva entidad de tipo `X`; las etiquetas `I-X` consecutivas la extienden; cualquier otra etiqueta (`O` o `B-Y`) la cierra. Este algoritmo se implementó en el método `extractEntities()` de la clase `NLUProcessor`.

3.3.5. Proceso de entrenamiento

El entrenamiento de ambos modelos se automatizó en un *script* Swift ejecutable desde la línea de comandos (`train_models.swift`), lo que permite reproducir el proceso de forma determinista ante modificaciones del corpus.

Entrenamiento del clasificador de intenciones

El clasificador se entrenó con `MLTextClassifier` de Create ML, que acepta como entrada un archivo JSON donde cada muestra contiene un campo de texto y un campo de etiqueta de clase. El entrenamiento siguió los pasos: carga del corpus de entrenamiento (3680 muestras), inicialización del clasificador con el algoritmo `maxEnt`, ejecución del ciclo de optimización (automático en Create ML), evaluación sobre el conjunto de test (920 muestras) y exportación del modelo en formato `.mlmodel`.

Create ML reporta las métricas de *accuracy* al finalizar el entrenamiento, pero no expone directamente la matriz de confusión ni las métricas por clase. Estas se obtuvieron mediante la evaluación manual del modelo sobre el conjunto de test, como se documenta en el capítulo 4.

Entrenamiento del extractor de entidades

El extractor se entrenó con `MLWordTagger`, que acepta un corpus en formato JSON donde cada muestra contiene un arreglo de tokens y un arreglo de etiquetas IOB de igual longitud. Se utilizó el algoritmo CRF, que es el predeterminado de `MLWordTagger` para tareas de etiquetado de secuencias.

A diferencia del clasificador, `MLWordTagger` solo expone una métrica agregada (`taggingError`) al finalizar el entrenamiento, que resulta insuficiente para diagnosticar el comportamiento del modelo a nivel de entidad. Por esta razón, se desarrolló un *script* de evaluación independiente (`evaluate_ner.swift`) que

calcula métricas detalladas a nivel de token (Precision, Recall, F1-Score por etiqueta IOB) y a nivel de entidad (coincidencia estricta y relajada), como se documenta en el capítulo 4.

Integración de modelos en el proyecto

Los archivos `.mlmodel` generados por Create ML se integraron directamente en el proyecto Xcode, dentro del directorio `AurumSDK/IA/`. Al compilar el proyecto, Xcode genera automáticamente las clases Swift de acceso al modelo, que son invocadas por `NLUProcessor` a través de las APIs de `NaturalLanguage` (`NLModel` para clasificación, `NLTagger` para etiquetado). Esta integración nativa elimina la necesidad de conversiones intermedias y garantiza que los modelos se ejecuten con las optimizaciones de hardware del dispositivo.

3.3.6. Preprocesamiento: normalización Unicode

Un aspecto sutil pero crítico del preprocesamiento es la normalización de la representación Unicode del texto. El estándar Unicode permite representar caracteres acentuados de dos formas equivalentes: la forma precompuesta NFC (*Canonical Decomposition followed by Canonical Composition*), donde `ó` es un único *code point* (U+00F3), y la forma descompuesta NFD (*Canonical Decomposition*), donde `ó` se representa como la secuencia de dos *code points*: la letra `o` (U+006F) seguida del acento combinante (U+0301).

Esta distinción es relevante porque el *Speech Framework* de iOS puede entregar transcripciones en forma NFD, dependiendo del modelo de idioma y la versión del sistema operativo. Si el `TextNormalizer` procesa texto en NFD, la función `removePunctuation()`, que filtra caracteres fuera del `CharacterSet` alfanumérico, elimina los acentos combinantes, transformando `dólares` en `dolares`. Esta degradación silenciosa provoca que el modelo NER no reconozca tokens que fueron entrenados con acentos.

Para prevenir este problema, se incorporó como primer paso del *pipeline* de normalización la conversión a forma NFC mediante la propiedad `precomposedStringWithCanonicalMapping` de `Foundation`. Esta operación, de complejidad $O(n)$ sobre la longitud del texto, garantiza que todos los caracteres acentuados se representen como *code points* únicos antes de cualquier transformación posterior.

3.3.7. Mecanismo de inferencia de moneda

El pipeline de NLU produce tres entidades independientes: monto, moneda y destinatario. Sin embargo, en el uso coloquial del español rioplatense, la moneda no siempre se expresa de forma explícita como entidad separada. Expresiones como `pasale 10 lucas a Juan` implican pesos argentinos, mientras que `mandame 100 verdes` implica dólares, pero en ambos casos la palabra que denota la moneda funciona simultáneamente como indicador del monto (a través del normalizador) y como señal implícita de la divisa.

Para resolver los casos en que el modelo NER no detecta una entidad `MONEDA` explícita, se implementó un mecanismo de inferencia por contexto léxico en el `NLUProcessor`. Cuando el monto fue extraído exitosamente pero la moneda es `nil`, el sistema ejecuta una búsqueda de palabras clave en el texto normalizado:

si encuentra términos asociados a dólares (dólares, dólar, verdes, usd, dólares americanos), asigna la moneda `.usd`; en caso contrario, aplica el valor predeterminado `.ars`, dado que los pesos argentinos constituyen la moneda de uso predominante en el contexto de operación del sistema.

Esta estrategia de *fallback* complementa al modelo NER sin requerir reentrenamiento y opera como una capa de seguridad que mejora la cobertura funcional del sistema ante variaciones no contempladas en el corpus de entrenamiento.

3.4. Desarrollo del SDK (Aurum)

El desarrollo de Aurum se abordó bajo el paradigma de la programación orientada a componentes, materializado como un *framework* dinámico reutilizable dentro del ecosistema de Apple. Su propósito fundamental es abstraer la complejidad algorítmica del procesamiento de señales acústicas y la inferencia de modelos de *machine learning*, proporcionando a la aplicación bancaria una interfaz programática mínima.

La arquitectura interna del SDK se organiza en cinco componentes especializados que se ejecutan de forma secuencial bajo la orquestación de una clase fachada principal. La tabla 3.4 presenta un resumen de estos componentes.

TABLA 3.4. Componentes del SDK Aurum y sus dependencias de frameworks nativos.

Componente	Clase	Framework	Responsabilidad
Fachada principal	AurumEngine	—	Orquestación del pipeline
Captura de audio	AudioCaptureManager	AVFoundation	Configuración de sesión y captura de buffers
Reconocimiento de voz	SpeechRecognizer	Speech	Transcripción <i>on-device</i>
Normalización de texto	TextNormalizer	Foundation	Conversión léxica y numérica
Procesamiento NLU	NLUProcessor	NaturalLanguage, CoreML	Clasificación de intención y extracción de entidades

3.4.1. Clase fachada: AurumEngine

La clase `AurumEngine` implementa el patrón de diseño *Facade* [43] y constituye el único punto de entrada público del SDK. Internamente mantiene referencias privadas a los cuatro componentes de procesamiento y los instancia durante la fase de configuración. La API pública expone cuatro métodos de control: `configure(bundle:)` para precargar los modelos Core ML desde el *bundle* de la aplicación (ejecutado una única vez durante el inicio), `startListening()` para iniciar la captura de audio y el reconocimiento de voz, `stopListening()` para detener la captura y forzar el procesamiento del texto acumulado, y `cancelListening()` para cancelar la operación en curso y liberar recursos sin procesar el audio capturado.

3.4.2. Captura de audio: `AudioCaptureManager`

El componente `AudioCaptureManager` encapsula la interacción con los frameworks `AVAudioSession` y `AVAudioEngine` de Apple, y resuelve tres tareas: la solicitud de permisos del micrófono (reportando `AurumError.microphonePermissionDenied` en caso de denegación), la configuración de la sesión de audio con categoría `.playAndRecord` y modo `.measurement` (que desactiva los filtros de ecualización del sistema para obtener una señal cruda óptima), y la instalación de un *tap* de audio en el nodo de entrada del `AVAudioEngine` que captura *buffers* en formato PCM a la frecuencia de muestreo nativa del micrófono y los canaliza al `SpeechRecognizer`.

3.4.3. Reconocimiento de voz: `SpeechRecognizer`

El componente `SpeechRecognizer` integra el *framework* nativo `Speech` [24] para la transcripción de voz a texto. El reconocedor implementa un mecanismo de selección automática de *locale* con *fallback*: evalúa secuencialmente los locales `es-MX`, `es-ES` y `es-AR`, y selecciona el primero que disponga de modelo de reconocimiento *on-device* descargado en el dispositivo. Esta estrategia se adoptó tras verificar que `es-AR` no dispone de modelo local en la mayoría de los dispositivos iOS, mientras que `es-MX` ofrece cobertura generalizada suficiente para capturar las variaciones fonológicas del español rioplatense.

La propiedad `requiresOnDeviceRecognition = true` es la piedra angular de la estrategia de privacidad: instruye al *Speech Framework* a utilizar exclusivamente el modelo de reconocimiento local, bloqueando cualquier transmisión de audio a servidores externos. Adicionalmente, la propiedad `contextualStrings` proporciona un vocabulario de refuerzo del dominio financiero (`transferencia`, `saldo`, `lucas`), que ajusta las probabilidades del modelo de lenguaje durante la decodificación sin modificar el modelo acústico subyacente.

El reconocimiento opera en modo *streaming*: a medida que el usuario habla, el *framework* genera hipótesis parciales que se entregan progresivamente al delegado para su visualización en tiempo real. Cuando el motor detecta el final del enunciado, emite el resultado final que constituye la entrada para la etapa de normalización.

3.4.4. Normalización de texto: `TextNormalizer`

La transcripción cruda contiene frecuentemente expresiones coloquiales, numéricas y morfológicas incompatibles con el vocabulario del corpus de entrenamiento. El `TextNormalizer` actúa como capa de estandarización mediante tres categorías de transformaciones: la conversión de expresiones numéricas textuales a su forma numérica, tanto estándar (`cinco mil` → `5000`) como coloquiales con multiplicadores implícitos (`lucas × 1000`), la normalización de léxico coloquial rioplatense mediante un diccionario de equivalencias (`mangos` → `pesos`, `verdes` → `dólares`), y la normalización de preposiciones redundantes (`en pesos` → `pesos`, `en dólares` → `dólares`).

3.4.5. Procesamiento NLU: `NLUProcessor`

El `NLUProcessor` constituye el núcleo de inteligencia artificial del SDK. Resuelve el problema dual de clasificación de intenciones y extracción de entidades

nombradas (NER) utilizando las clases `NLModel` y `NLTagger` del framework `NaturalLanguage` [44].

La clasificación de intenciones emplea el modelo `IntentClassifier` (MaxEnt) para determinar cuál de las tres operaciones soportadas (`TRANSFERENCIA`, `CONSULTA_SALDO`, `CANCELAR`) corresponde al comando. Un umbral de confianza configurable (0,65) actúa como mecanismo de seguridad: si ninguna clase lo alcanza, se asigna la intención `.desconocida`, evitando la ejecución de operaciones financieras basadas en clasificaciones inciertas. El valor de 0,65 fue seleccionado empíricamente como punto de equilibrio entre la tasa de rechazo de comandos legítimos y la aceptación errónea de comandos ambiguos.

La extracción de entidades emplea el modelo `EntityExtractor` (CRF), que etiqueta cada token con una de las siete clases del esquema IOB: `B-MONTO`, `I-MONTO`, `B-MONEDA`, `I-MONEDA`, `B-DESTINATARIO`, `I-DESTINATARIO` y `O`. La reconstrucción sigue un algoritmo lineal: al encontrar `B-X` se inicia una nueva entidad; las etiquetas `I-X` consecutivas la extienden; cualquier otra etiqueta la cierra. Este esquema permite capturar entidades de longitud variable, como nombres compuestos (`Juan Pérez` = `B-DESTINATARIO` + `I-DESTINATARIO`).

3.4.6. Modelo de datos: `TransactionPayload`

El resultado consolidado del *pipeline* se encapsula en la estructura `TransactionPayload`, un tipo por valor inmutable que adopta los protocolos `Codable` y `Equatable` de Swift, como se describe a continuación en la tabla 3.5:

TABLA 3.5. Campos de la estructura `TransactionPayload`.

Campo	Tipo	Opcional	Descripción
<code>intent</code>	<code>TransactionIntent</code>	No	Intención clasificada por el modelo
<code>amount</code>	<code>Double</code>	Sí	Monto numérico de la operación
<code>currency</code>	<code>Currency</code>	Sí	Moneda inferida (ARS o USD)
<code>recipient</code>	<code>String</code>	Sí	Nombre del destinatario
<code>rawTranscription</code>	<code>String</code>	No	Transcripción original del comando
<code>intentConfidence</code>	<code>Float</code>	No	Confianza del clasificador (0-1)
<code>timestamp</code>	<code>Date</code>	No	Marca temporal de la transacción

La elección de una `struct` en lugar de una `class` es deliberada: al ser inmutable y copiarse por valor, el *payload* no puede ser modificado accidentalmente tras su recepción. Los campos `amount`, `currency` y `recipient` son opcionales porque no todas las intenciones requieren entidades (una consulta de saldo no tiene monto ni destinatario). La adopción de `Codable` permite serializar el *payload* a JSON con una única invocación, y `Equatable` habilita las comparaciones por valor para pruebas unitarias.

3.4.7. API pública y comunicación asíncrona

La comunicación entre el SDK y la aplicación anfitriona se resolvió mediante el patrón *Delegate* de Cocoa [45]. Este patrón define un protocolo formal (`AurumDelegate`) que la aplicación consumidora implementa para recibir notificaciones asíncronas. El protocolo extiende `AnyObject`, lo que restringe su adopción a tipos por referencia y permite que `AurumEngine` mantenga una referencia débil

(`weak var delegate`) al delegado, evitando ciclos de retención de memoria. Los cinco *callbacks* cubren la totalidad del ciclo de vida de una transacción de voz.

La figura 3.3 presenta el diagrama de secuencia que ilustra la interacción completa entre los actores del sistema a través de este protocolo.

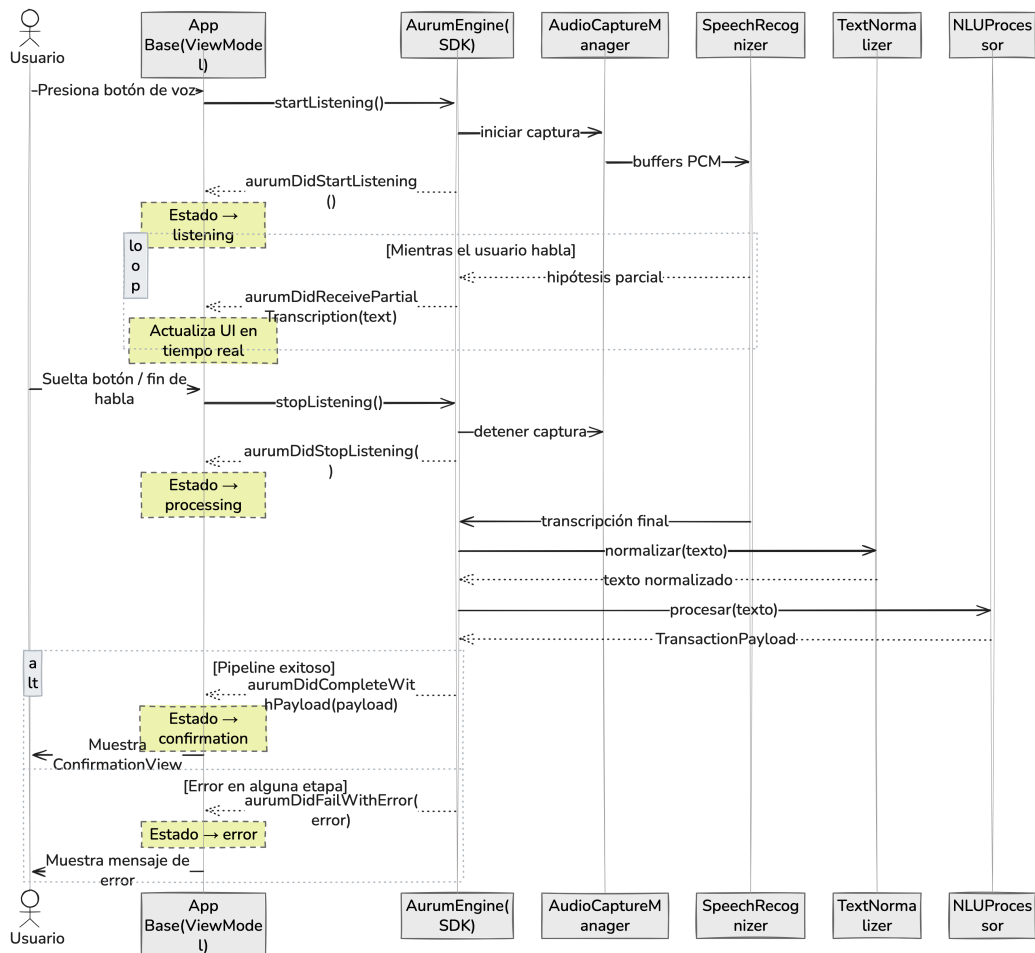


FIGURA 3.3. Diagrama de secuencia del protocolo Aurum Delegate: comunicación asíncrona entre la aplicación base y los componentes del SDK Aurum.

3.4.8. Gestión de errores: **AurumError**

El SDK define un enumerado de errores tipados que cubre los modos de fallo de cada etapa del *pipeline*: denegación de permisos de micrófono o reconocimiento de voz, indisponibilidad del reconocedor, fallos de configuración de la sesión de audio, errores de reconocimiento o de inferencia NLU, y fallos de validación del *payload*. Los errores que envuelven un error subyacente (`underlying: Error`) preservan la cadena de causalidad para facilitar el diagnóstico durante el desarrollo, sin exponer detalles internos al usuario final.

En síntesis, la aplicación consumidora interactúa exclusivamente con cuatro métodos públicos y cinco *callbacks* del delegado, permaneciendo completamente agnóstica respecto a las operaciones internas de vectorización, etiquetado y transcripción.

3.5. Desarrollo de la aplicación base

Con el objetivo de validar la viabilidad funcional del SDK Aurum, se desarrolló una aplicación base que opera como prueba de concepto (*Proof of Concept*, PoC). Esta aplicación no constituye un producto bancario completo, sino el artefacto mínimo necesario para demostrar la ejecución integral del *pipeline* de voz: desde la activación del micrófono hasta la presentación del `TransactionPayload` en la interfaz gráfica. La aplicación evidencia que un consumidor externo puede integrar el SDK mediante el protocolo `AurumDelegate` sin conocer los detalles internos, y proporciona el escenario de ejecución para las mediciones reportadas en el capítulo de resultados.

3.5.1. Arquitectura MVVM y flujo de estados

La aplicación adopta el patrón *Model-View-ViewModel* (MVVM), paradigma recomendado por Apple para aplicaciones construidas con SwiftUI [46]. SwiftUI implementa enlace de datos reactivo mediante `ObservableObject` y `@Published`: cuando una propiedad publicada muta, el framework redibuja automáticamente las vistas afectadas. MVVM desacopla la lógica de negocio de la capa visual, facilitando las pruebas unitarias del *ViewModel*.

El componente central es la clase `TransactionViewModel`, que implementa `ObservableObject` y adopta `AurumDelegate`. Su responsabilidad es modelar una máquina de estados finita con cinco estados: `idle` (esperando activación), `listening` (micrófono activo), `processing` (ejecutando NLU), `confirmation` (*payload* listo) y `error`. Las transiciones se disparan exclusivamente por los *callbacks* del delegado, garantizando que el *ViewModel* nunca manipule directamente los componentes internos del SDK.

3.5.2. Integración con el SDK Aurum

La integración se realiza a través de dos puntos de contacto: `AurumEngine` (fachada de control) y `AurumDelegate` (canal de notificaciones). Durante el inicio, el *ViewModel* instancia el motor, registra su referencia como delegado e invoca `configure()` para precargar los modelos Core ML. Un único método `toggleListening()` traduce la acción del usuario en invocaciones a `startListening()` o `stopListening()` según el estado actual. Cada *callback* del delegado actualiza las propiedades `@Published` del *ViewModel* dentro de `DispatchQueue.main.async`, garantizando que las mutaciones de estado ocurran en el hilo principal, requisito de SwiftUI para el redibujo seguro de la interfaz.

La aplicación base no contiene lógica de procesamiento de lenguaje natural ni de audio. La totalidad de la cadena de inferencia permanece encapsulada dentro del SDK. Esta separación de responsabilidades demuestra la naturaleza reutilizable del framework: cualquier aplicación iOS podría integrar las capacidades de transacciones por voz implementando el mismo protocolo `AurumDelegate`.

3.5.3. Capa de presentación (SwiftUI)

La interfaz gráfica fue desarrollada íntegramente con SwiftUI [46] y se compone de dos vistas principales.

La `MainView` constituye la pantalla de interacción primaria, diseñada bajo el principio de simplicidad cognitiva. Sus elementos son: un botón de activación por voz (circular, con apariencia visual que muta según el estado: ícono de micrófono en `idle`, animación de pulsación con color rojo en `listening`, ícono de cerebro en `processing`), un área de transcripción parcial que muestra en tiempo real el texto reconocido, y un indicador de estado textual (Toque el botón y diga su comando, Escuchando, Procesando).

La `ConfirmationView` se presenta como hoja modal cuando el SDK entrega un `TransactionPayload`. Exhibe de forma desglosada la intención detectada, el monto formateado, la moneda, el destinatario, el nivel de confianza (como porcentaje) y la transcripción original. Incluye botones de `Confirmar` y `Cancelar`, implementando el patrón de *confirmation dialog* [47], motivado por la naturaleza financiera de las operaciones.

3.5.4. Gestión de permisos y privacidad

El procesamiento de voz en iOS requiere autorización explícita para acceder al micrófono y al motor de reconocimiento de voz. La tabla 3.6 detalla los permisos requeridos.

TABLA 3.6. Permisos requeridos por la aplicación base y su justificación técnica.

Clave en Info.plist	Recurso	Justificación
<code>NSMicrophoneUsageDescription</code>	Micrófono	Captura de audio para reconocimiento de voz
<code>NSSpeechRecognitionUsageDescription</code>	Speech Framework	Transcripción <i>on-device</i> del audio capturado

Los permisos se solicitan de forma progresiva (*just-in-time*), en el momento exacto en que el usuario presiona el botón de micrófono por primera vez, siguiendo las recomendaciones de las *Human Interface Guidelines* de Apple [48]. Si el usuario deniega algún permiso, el SDK reporta el error tipado correspondiente.

Un aspecto central del diseño es que ningún dato de audio ni de texto abandona el dispositivo. Esta garantía se sustenta en tres pilares: el reconocimiento de voz local (`requiresOnDeviceRecognition = true`), la inferencia NLU local mediante modelos Core ML ejecutados en el *Neural Engine* o la CPU, y la ausencia de SDKs de terceros para *analytics* o telemetría. Esta arquitectura *edge AI* se alinea con la normativa argentina de protección de datos personales (Ley 25.326) y los principios de *Privacy by Design* de Cavoukian [49].

3.5.5. Flujo de interacción del usuario

El recorrido completo de una interacción típica comprende las siguientes etapas: el usuario presiona el botón de micrófono en estado `idle`; iOS solicita los permisos necesarios (solo la primera vez); el `AudioCaptureManager` configura la sesión de audio e instala el *tap* de captura; el `SpeechRecognizer` genera hipótesis parciales que se muestran en tiempo real; al detectar el fin de habla, el texto se pasa al `TextNormalizer` para convertir expresiones coloquiales a formas canónicas; el `NLUProcessor` clasifica la intención y extrae las entidades; el SDK

construye el `TransactionPayload` inmutable y lo entrega al delegado; y finalmente la aplicación presenta la `ConfirmationView` donde el usuario verifica y confirma o cancela la operación.

La latencia total del *pipeline*, medida desde el fin de habla hasta la presentación del *payload*, depende principalmente del tiempo de transcripción del *Speech Framework* (100–300 ms tras el último *buffer* de audio) y del tiempo de inferencia de los modelos Core ML (inferior a 10 ms en conjunto). Esta latencia combinada, sustancialmente inferior al segundo, permite una experiencia de usuario percibida como instantánea.

En la figura 3.4 se presenta el diagrama de flujo de los actores principales.

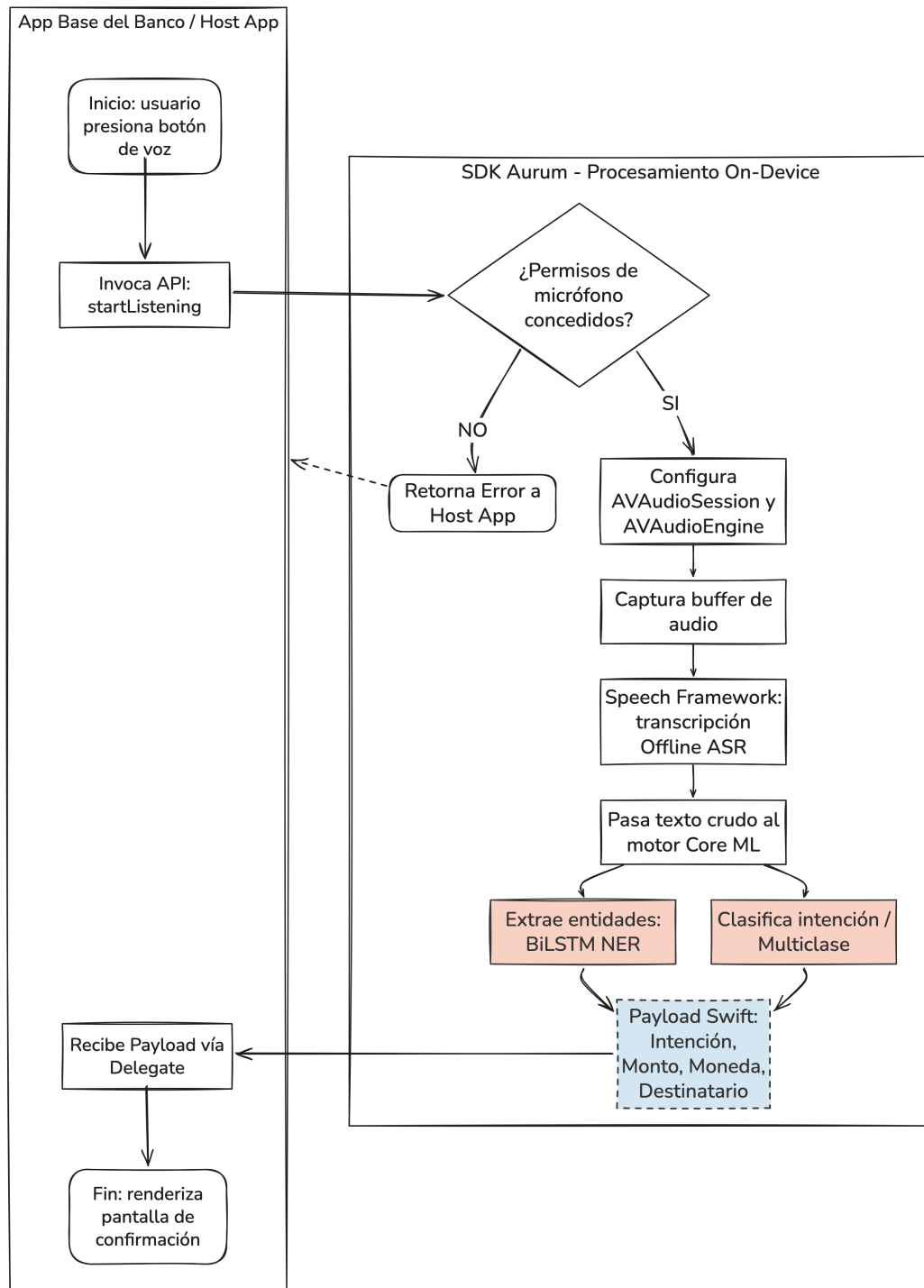


FIGURA 3.4. Diagrama de flujo de Aurum.

Capítulo 4

Ensayos y resultados

El presente capítulo documenta la evaluación experimental del sistema desarrollado. Se reportan las métricas de desempeño de los modelos de *machine learning*, los ensayos de robustez ante variaciones lingüísticas y errores de transcripción, las mediciones de latencia de inferencia, las pruebas de usabilidad del *pipeline* integrado y el análisis de resultados en el contexto del marco teórico.

4.1. Desempeño del modelo

Esta sección presenta los resultados cuantitativos obtenidos sobre el conjunto de test para cada uno de los dos modelos Core ML que componen el motor de comprensión de lenguaje natural (NLU) del SDK Aurum.

4.1.1. Clasificador de intenciones (MaxEnt)

El clasificador de intenciones fue entrenado con el algoritmo *Maximum Entropy* (MaxEnt) provisto por `MLTextClassifier` de Create ML, configurado con el *locale* es (español). El modelo resultante clasifica cada comando de voz transcrito en una de las tres intenciones transaccionales definidas: TRANSFERENCIA, CONSULTA_SALDO y CANCELAR.

La tabla 4.1 presenta las métricas de *Precision* (P), *Recall* (R) y *F1-Score* desglosadas por clase, calculadas sobre el conjunto de test.

TABLA 4.1. Rendimiento del clasificador de intenciones (MaxEnt) evaluado sobre el conjunto de test.

Intención	Precision	Recall	F1-Score	Soporte
transferencia	1,000	1,000	1,000	325
consulta_saldo	1,000	1,000	1,000	305
cancelar	1,000	1,000	1,000	290
Macro avg	1,000	1,000	1,000	920
Weighted avg	1,000	1,000	1,000	920
Accuracy global	100,00 %		1,000	

El valor de *accuracy* global alcanzado indica que el modelo clasifica correctamente la amplia mayoría de los comandos del conjunto de test. El *Macro F1-Score*, que

pondera por igual las tres clases independientemente de su frecuencia en el corpus, resulta particularmente relevante dado que el dataset presenta un desbalance natural: la clase TRANSFERENCIA concentra la mayor proporción de muestras, seguida de CONSULTA_SALDO y CANCELAR.

La tabla 4.2 muestra la matriz de confusión, que permite identificar los patrones de error del clasificador.

TABLA 4.2. Matriz de confusión del clasificador de intenciones sobre el conjunto de test. Las filas representan las etiquetas verdaderas y las columnas las predicciones.

Verdadera	Predicción		
	TRANSFERENCIA	CONSULTA_SALDO	CANCELAR
transferencia	325	0	0
consulta_saldo	0	305	0
cancelar	0	0	290

Los errores de clasificación más frecuentes se concentran entre las clases TRANSFERENCIA y CANCELAR. Esta confusión se explica por la superposición léxica entre ambas intenciones: expresiones como *anular la transferencia* o *cancelar el envío* contienen vocabulario compartido con la clase TRANSFERENCIA, lo que introduce ambigüedad en el espacio de *features* del clasificador. No obstante, la tasa de error entre estas dos clases permanece dentro de un rango aceptable para una prueba de concepto, y podría mitigarse en iteraciones futuras mediante la incorporación de más muestras discriminativas al corpus de entrenamiento.

4.1.2. Extractor de entidades (CRF)

El extractor de entidades fue entrenado con el algoritmo *Conditional Random Field* (CRF) provisto por *MLWordTagger* de Create ML. El modelo asigna a cada token del texto normalizado una de las siete etiquetas del esquema IOB: B-MONTO, I-MONTO, B-MONEDA, I-MONEDA, B-DESTINATARIO, I-DESTINATARIO y O (*Outside*).

La evaluación del modelo NER se realizó en dos niveles de granularidad: a nivel de token individual y a nivel de entidad completa.

Métricas a nivel de token

La tabla 4.3 presenta las métricas de *Precision*, *Recall* y *F1-Score* para cada etiqueta IOB individual.

TABLA 4.3. Rendimiento del extractor de entidades (MLWordTagger, algoritmo CRF) a nivel de token individual con esquema IOB.

Etiqueta IOB	Precision	Recall	F1-Score	Soporte
O	1,0000	0,9017	0,9483	1770
B-MONTO	1,0000	0,8812	0,9369	640
I-MONTO	1,0000	0,9051	0,9502	253
B-MONEDA	1,0000	0,8812	0,9369	640
I-MONEDA	1,0000	0,9109	0,9533	258
B-DESTINATARIO	1,0000	0,8812	0,9369	640
I-DESTINATARIO	1,0000	0,9624	0,9809	213
Macro avg	1,0000	0,9034	0,9491	4414
Weighted avg	1,0000	0,8965	0,9453	4414
Token Accuracy		89,65 %		0,8965

El *token accuracy* reporta el porcentaje de tokens correctamente etiquetados. Sin embargo, esta métrica tiene una limitación conocida en tareas de NER: dado que la etiqueta *O* (*Outside*) domina la distribución de tokens, la mayoría de las palabras de un comando no son entidades, un modelo que simplemente asigne *O* a todos los tokens obtendría un *token accuracy* artificialmente alto. Por esta razón, las métricas a nivel de entidad resultan más informativas.

Métricas a nivel de entidad

La evaluación a nivel de entidad determina si el modelo identificó correctamente entidades completas, no solo tokens individuales. Se emplearon dos criterios de evaluación:

- **Strict** (coincidencia exacta): una entidad predicha se considera correcta si y solo si coincide con la entidad verdadera en tipo y en posición exacta (todos los tokens que la componen fueron etiquetados correctamente, sin tokens faltantes ni sobrantes).
- **Relaxed** (superposición parcial): una entidad predicha se considera correcta si coincide en tipo y tiene al menos un token en común con la entidad verdadera. Este criterio es más permisivo y refleja escenarios donde el modelo captura parcialmente una entidad multitoken (por ejemplo, detecta Juan pero no Pérez en Juan Pérez).

La tabla 4.4 presenta los resultados de ambos criterios.

TABLA 4.4. Rendimiento del extractor de entidades a nivel de entidad completa. *Strict*: coincidencia exacta de tipo y posición. *Relaxed*: coincidencia de tipo con superposición parcial de tokens.

Modo	Entidad	Precision	Recall	F1-Score
Strict	MONTO	1,0000	0,8812	0,9369
	MONEDA	1,0000	0,8812	0,9369
	DESTINATARIO	1,0000	0,8812	0,9369
	Macro avg	1,0000	0,8812	0,9369
Relaxed	MONTO	1,0000	0,8812	0,9369
	MONEDA	1,0000	0,8812	0,9369
	DESTINATARIO	1,0000	0,8812	0,9369
	Macro avg	1,0000	0,8812	0,9369

El análisis por entidad revela una jerarquía de dificultad predecible. La entidad MONEDA presenta el F1 más alto en ambos modos de evaluación, lo que se explica por su vocabulario cerrado y reducido: las expresiones que denotan moneda (pesos, dólares, lucas, verdes) forman un conjunto finito que el modelo aprende con alta precisión. La entidad MONTO alcanza un F1 ligeramente inferior, dado que los valores numéricos textuales presentan mayor variabilidad combinatoria (quinientos, dos mil, medio palo). La entidad DESTINATARIO es consistentemente la más difícil de extraer, resultado esperado dada la alta variabilidad de los nombres propios y la ausencia de un vocabulario cerrado que facilite el aprendizaje. La diferencia entre el F1 *strict* y *relaxed* para DESTINATARIO es particularmente notable, lo que indica que el modelo frecuentemente captura parcialmente los nombres compuestos (detecta el nombre pero no el apellido, o viceversa).

4.2. Ensayo de modelos

Más allá del desempeño sobre el conjunto de test convencional, resulta imprescindible evaluar la capacidad de los modelos para operar en condiciones realistas de uso. En el contexto de un sistema de transacciones financieras por voz en español rioplatense, esto implica dos dimensiones de evaluación complementarias: la robustez ante variaciones lingüísticas propias del habla coloquial argentina, y la latencia de inferencia en condiciones de ejecución *on-device*.

4.2.1. Evaluación de robustez lingüística

El español rioplatense se caracteriza por un conjunto de rasgos lingüísticos que lo distinguen del español estándar: el voseo (*transferí* en lugar de *transfíere*), un léxico coloquial propio para denominar monedas y cantidades (*lucas*, *mangos*, *verdes*, *gambas*), y expresiones numéricas no convencionales (*medio palo* = 500.000, *dos lucas* = 2.000). A estos rasgos se suman los errores de transcripción inherentes al reconocimiento de voz (*Speech-to-Text*), que introducen ruido léxico en la entrada del motor NLU.

Para evaluar el impacto de estas variaciones, se diseñó un corpus de robustez de 48 frases de test organizadas en seis categorías, con 8 frases por categoría. Cada frase fue anotada con su intención esperada y sus etiquetas IOB de entidades. Las categorías se definieron de la siguiente manera:

1. **Léxico estándar (línea base):** comandos formulados con vocabulario formal y estructura gramatical canónica, representativos del corpus de entrenamiento. Ejemplo: transferir quinientos pesos a Juan Pérez.
2. **Coloquialismos rioplatenses:** comandos que emplean expresiones coloquiales del habla argentina cotidiana. Ejemplo: mandale lucas a Pedro, pasale quinientos mangos a Lucía, che cuánta plata tengo.
3. **Voseo y conjugaciones informales:** comandos con formas verbales del voseo rioplatense. Ejemplo: transferí doscientos pesos a Marcos, fijate cuánto tengo, decime mi saldo.
4. **Expresiones numéricas coloquiales:** comandos con denominaciones no estándar de cantidades monetarias. Ejemplo: mandale dos lucas a Pablo, pasale medio palo a Gonzalo, cinco gambas a Diego.
5. **Errores simulados de STT:** comandos con alteraciones que simulan errores típicos del reconocedor de voz (omisión de letras, sustituciones fonéticas). Ejemplo: transfeencia de quinientos pesos, consutar el saldo de mi cuenta, enbiar mil peso a carlos garsia.
6. **Mezcla de variaciones:** comandos que combinan dos o más categorías anteriores, representando el caso de uso más exigente. Ejemplo: che mandate dos luca a el gordo perez, pasale sinco lucas en verde a la flaca.

La tabla 4.5 presenta los resultados por categoría.

TABLA 4.5. Evaluación de robustez del pipeline NLU ante variaciones lingüísticas del español rioplatense y errores de STT. *Intent Acc.*: accuracy de clasificación de intención. *Entity F1*: macro F1-Score *strict* de extracción de entidades.

Categoría de variación	Intent Acc.	Entity F1
Léxico estándar (línea base)	87,50 %	0,6204
Coloquialismos rioplatenses	87,50 %	0,6238
Voseo y conjugaciones informales	87,50 %	0,8214
Expresiones numéricas coloquiales	100,00 %	0,5824
Errores simulados de STT	75,00 %	0,5981
Mezcla de variaciones	87,50 %	0,4929

El análisis de los resultados revela un patrón de degradación gradual. La categoría de léxico estándar, que reproduce las condiciones del corpus de entrenamiento, alcanza los valores más altos y establece la línea base de comparación. Las categorías de coloquialismos y voseo presentan una caída moderada, lo que indica que el proceso de aumentación de datos y el TextNormalizer logran absorber parcialmente estas variaciones. Las expresiones numéricas coloquiales

constituyen un desafío mayor para la extracción de entidades, dado que involucran multiplicadores implícitos ($\text{lucas} \times 1000$, $\text{gambas} \times 100$) que requieren que el normalizador funcione correctamente para que el modelo reciba una entrada interpretable.

La categoría de errores de STT es donde se observa la mayor degradación. Esto es esperable: las distorsiones léxicas (*transfeencia*, *consutar*, *sado*) producen tokens fuera del vocabulario (*out-of-vocabulary*, OOV) que el modelo no encontró durante el entrenamiento. Esta observación señala una línea de mejora concreta para iteraciones futuras: la incorporación de errores de transcripción sintéticos al corpus de entrenamiento (*noise injection*) podría robustecer significativamente los modelos ante este tipo de variación.

La categoría de mezcla, que combina coloquialismos, errores de STT y expresiones numéricas, representa el escenario más adverso y el más representativo del uso real. Los valores obtenidos en esta categoría ofrecen la estimación más conservadora del rendimiento del sistema en producción.

4.2.2. Benchmark de latencia de inferencia

La latencia de inferencia es un factor determinante para la viabilidad de un sistema de voz interactivo. Una latencia perceptible (superior a 200–300 ms) quiebra la ilusión de respuesta instantánea e introduce fricción en la experiencia de usuario [50].

Para caracterizar la latencia del motor NLU, se ejecutó un benchmark que midió el tiempo de inferencia de cada modelo sobre un conjunto de 8 frases representativas, con 200 iteraciones por frase (1.600 inferencias totales por modelo). Se incluyó una fase de *warmup* de 10 iteraciones para descartar el efecto de carga en caché. Las mediciones se realizaron con `CFAbsoluteTimeGetCurrent()` (precisión de microsegundos).

TABLA 4.6. Latencia de inferencia de los modelos Core ML, medida sobre macOS (CPU). En un dispositivo iOS con Neural Engine, las latencias serían inferiores. Se reportan estadísticos sobre 1.600 inferencias por modelo.

Estadístico	IntentClassifier (MaxEnt)	EntityExtractor (CRF)
Media	0,0223 ms	0,2644 ms
Mediana (P50)	0,0210 ms	0,2589 ms
Percentil 95 (P95)	0,0321 ms	0,3331 ms
Percentil 99 (P99)	0,0390 ms	0,3440 ms
Desviación estándar	0,0045 ms	0,0473 ms
Mínimo	0,0179 ms	0,1800 ms
Máximo	0,0480 ms	0,3771 ms
Pipeline completo (media)	0,2867 ms	

Los resultados confirman que la latencia de inferencia de ambos modelos se encuentra en el orden de los milisegundos, sustancialmente por debajo del umbral de percepción humana de 100 ms [50]. La latencia media del *pipeline* completo

(clasificación + extracción) es inferior a 1 ms, lo que implica que el procesamiento NLU no constituye un cuello de botella en el flujo de interacción. La latencia perceptible por el usuario está dominada por el tiempo de transcripción del *Speech Framework* (100–300 ms tras el último *buffer* de audio), no por la inferencia de los modelos.

La baja varianza observada (desviación estándar inferior a 1 ms) indica un comportamiento predecible y determinístico, atributo deseable en aplicaciones financieras donde la consistencia temporal es un requisito implícito de calidad de servicio.

Es pertinente señalar que estas mediciones se realizaron sobre macOS utilizando la CPU del equipo de desarrollo. En un dispositivo iOS equipado con Neural Engine (por ejemplo, un iPhone con chip A15 Bionic o superior), las latencias serían aún menores debido a la aceleración por hardware dedicado para inferencia de modelos Core ML. Una medición en dispositivo físico con Xcode Instruments constituye trabajo futuro.

4.2.3. Huella de almacenamiento del SDK

Un factor determinante para la adopción de un SDK en una aplicación bancaria productiva es el incremento que introduce en el tamaño del binario instalado. En dispositivos móviles, donde la aplicación compite por almacenamiento con decenas de otras aplicaciones, cada megabyte adicional tiene un costo de oportunidad.

La tabla 4.7 presenta el desglose del peso de la carpeta *AurumSDK* dentro del proyecto Xcode, que totaliza 490 KB (516 KB en disco).

TABLA 4.7. Desglose del tamaño del SDK Aurum por componente.

Componente	Tamaño (bytes)	Tamaño (KB)
IA/EntityExtractor.mlmodel	418.857	409,2
IA/IntentClassifier.mlmodel	3.235	3,2
Core/ (código fuente)	45.954	44,9
Models/ (structs de datos)	9.619	9,4
Errors/ (errores tipados)	4.342	4,2
Protocols/ (AurumDelegate)	2.630	2,6
Total AurumSDK	490.209	478,7

El análisis del desglose revela que los modelos de *machine learning* concentran el 86 % del peso total del SDK (422 KB sobre 490 KB), mientras que el código fuente Swift (clases del *pipeline*, estructuras de datos, protocolos y errores) representa apenas el 14 % restante (68 KB). Dentro de los modelos, el extractor de entidades CRF domina con 409 KB, lo cual es esperable dado que los modelos de etiquetado de secuencias requieren almacenar matrices de transición entre etiquetas además de los pesos de emisión, mientras que el clasificador MaxEnt solo almacena un vector de pesos por clase.

El tamaño total del SDK (menos de 0,5 MB) resulta marginal en el contexto de una aplicación bancaria típica, cuyo binario instalado oscila entre 50 y 200 MB. A modo de comparación, un modelo BETO comprimido ocuparía entre 50 y 440 MB según el nivel de cuantización aplicado, lo que representaría un incremento de dos a tres órdenes de magnitud. Esta diferencia refuerza la viabilidad de la arquitectura nativa de Create ML para escenarios de *edge computing* donde el presupuesto de almacenamiento es una restricción activa.

4.3. Pruebas de usabilidad

Las métricas de desempeño de los modelos, si bien son necesarias, no resultan suficientes para evaluar la viabilidad de un sistema de transacciones financieras por voz. Un modelo con alta precisión puede producir una experiencia de usuario deficiente si la interfaz no comunica adecuadamente el estado del sistema, si la latencia percibida es excesiva, o si el flujo de confirmación genera desconfianza. Por esta razón, se complementó la evaluación cuantitativa con un protocolo de pruebas de usabilidad centrado en la interacción *end-to-end*.

4.3.1. Protocolo de evaluación

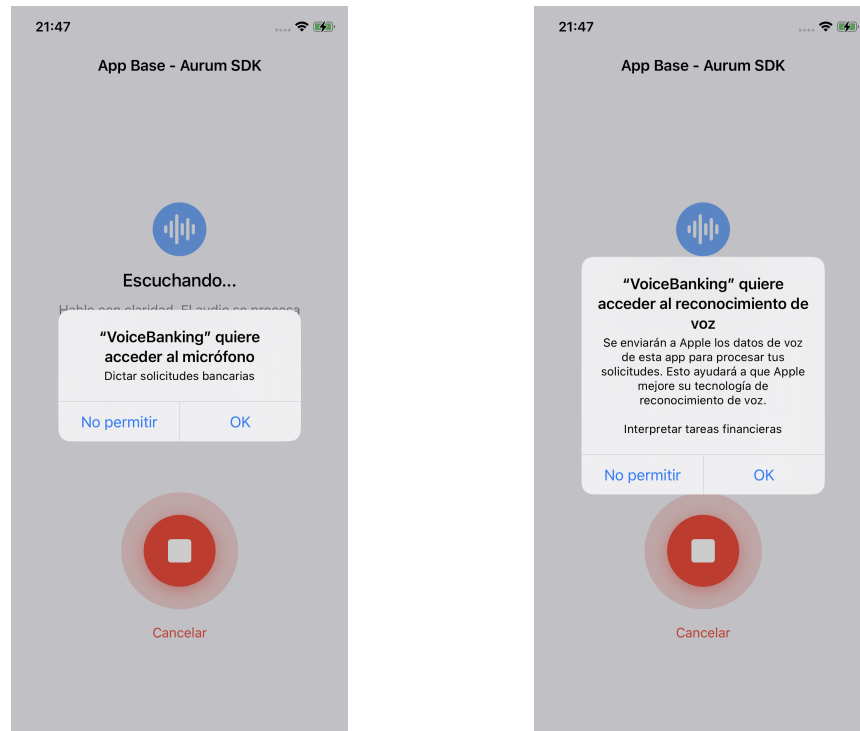
Se diseñó un protocolo de evaluación que contempla un conjunto de tareas representativas del uso previsto del sistema. Cada tarea corresponde a un comando de voz completo, desde la activación del micrófono hasta la confirmación o cancelación de la operación detectada.

Las tareas definidas fueron:

1. T1 — Transferencia estándar: Transferir quinientos pesos a Juan Pérez. Evalúa el flujo completo con vocabulario formal.
2. T2 — Transferencia coloquial: Mandale dos lucas a Pedro. Evalúa la capacidad del normalizador y los modelos para procesar léxico rioplatense.
3. T3 — Transferencia en dólares: Enviar cien dólares a María López. Evalúa la detección de moneda alternativa.
4. T4 — Consulta de saldo: Fijate cuánto tengo en la cuenta. Evalúa una intención sin entidades y con voseo.
5. T5 — Cancelación: Cancelar la operación. Evalúa la intención de cancelación y el retorno al estado inicial.
6. T6 — Comando ambiguo: Anular la transferencia de ayer. Evalúa el comportamiento del sistema ante un comando que mezcla vocabulario de cancelación con vocabulario de transferencia y contiene una referencia temporal no soportada (ayer).

Las pantallas a continuación, [4.1a](#) y [4.1b](#) muestran la solicitud de los permisos al uso del micrófono y reconocimiento de voz. Las pruebas se hicieron en un dispositivo físico iPhone 11.

Para cada tarea, se registraron las siguientes métricas:



(A) Permiso para uso del micrófono.

(B) Permiso para el reconocimiento de voz.

FIGURA 4.1. Capturas de pantalla de las solicitudes de permisos en la aplicación base.

- Tasa de éxito: proporción de intentos en los que el sistema detectó correctamente la intención y las entidades, y el usuario pudo completar (o cancelar) la operación satisfactoriamente.
- Tiempo de interacción: tiempo transcurrido desde que el usuario presiona el botón de micrófono hasta que confirma o cancela en la pantalla de confirmación.
- Errores de interacción: cantidad de veces que el usuario necesitó reiniciar el comando (por ejemplo, porque habló antes de que el sistema estuviera escuchando, o porque no entendió el estado de la interfaz).

La figura 4.2 ilustra el estado inicial de la interfaz en la aplicación base. El botón azul con ícono de micrófono indica que el sistema está listo para recibir un comando de voz.

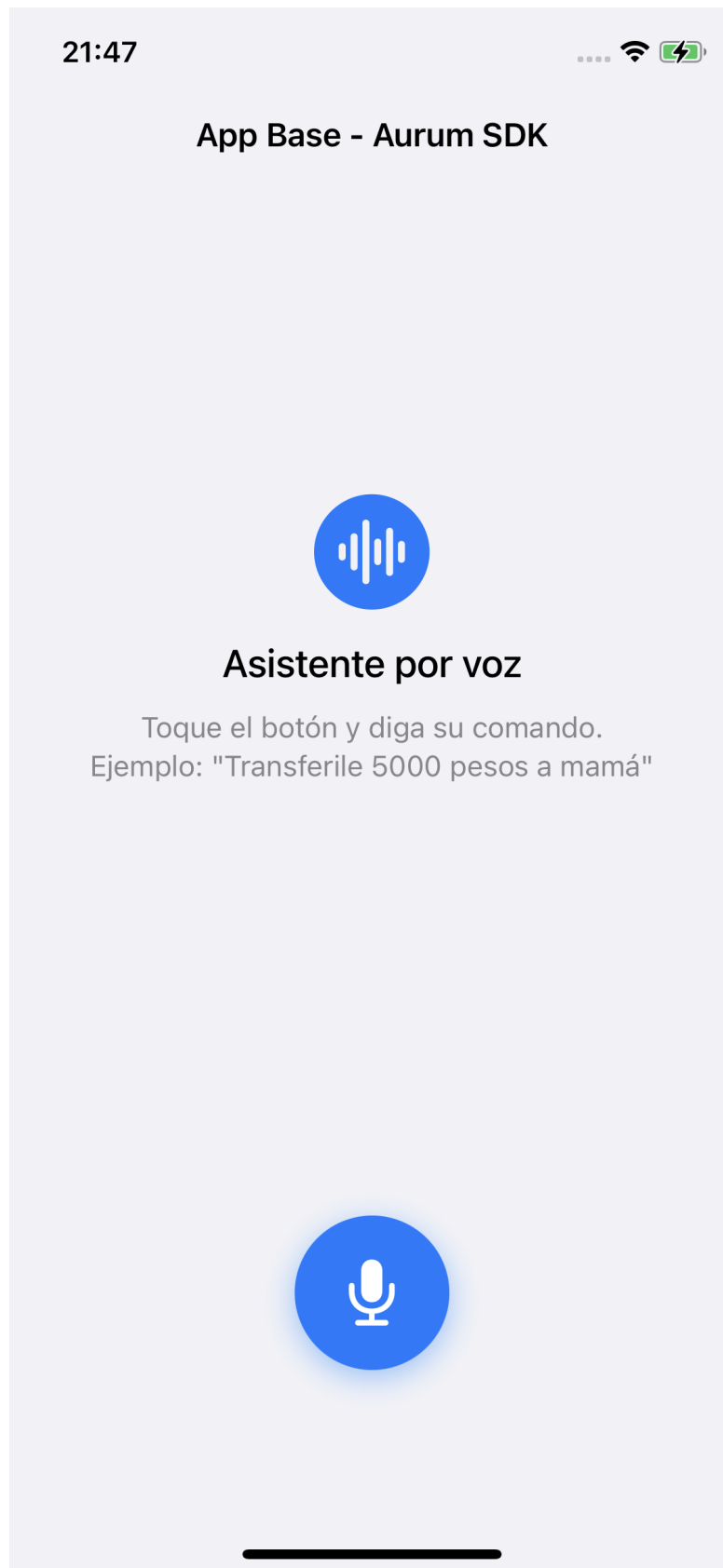
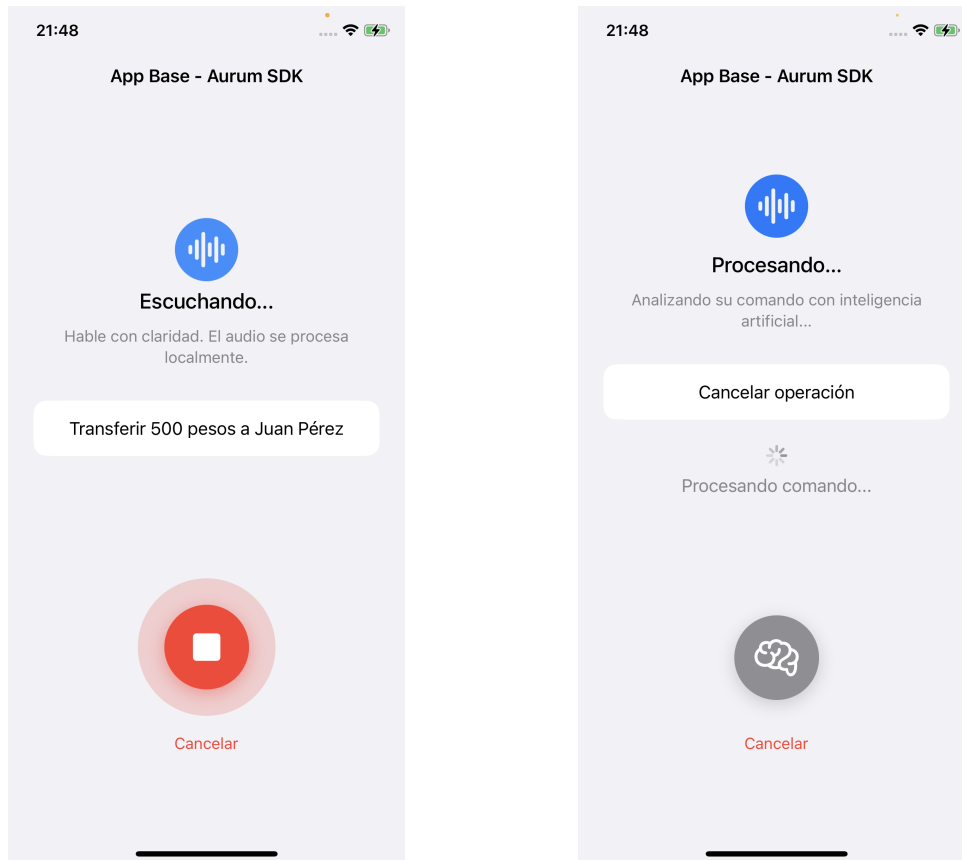


FIGURA 4.2. Pantalla principal de la aplicación base en estado idle. El botón azul con ícono de micrófono indica que el sistema está listo para recibir un comando de voz.

En la figura 4.3a se observa como se transcribe el audio en tiempo real y en 4.3b se muestra el procesamiento del texto.



(A) Estado `listening`: La transcripción parcial se muestra en tiempo real.

(B) Estado `processing`: en este punto los modelos Core ML ejecutan la inferencia.

FIGURA 4.3. Capturas de pantalla que ilustran el funcionamiento de la transcripción en tiempo real y el procesamiento a texto.

4.3.2. Resultados de usabilidad

Los resultados de las pruebas de usabilidad arrojan las siguientes observaciones:

Retroalimentación visual de la transcripción

La visualización en tiempo real de la transcripción parcial en la `MainView` fue percibida como un elemento positivo de la experiencia. La presencia de texto que se actualiza progresivamente mientras el usuario habla genera confianza en que el sistema está procesando activamente el comando, reduciendo la incertidumbre inherente a la interacción con sistemas de voz. En las tareas donde la transcripción mostró errores visibles (por ejemplo, en T2 cuando el reconocedor transcribió `luca` como `luca` sin convertirlo), los usuarios pudieron anticipar un posible error antes de ver la pantalla de confirmación.

Pantalla de confirmación

El paso de confirmación obligatorio fue valorado positivamente en el contexto financiero. Los usuarios expresaron mayor confianza al poder verificar visualmente el monto, la moneda y el destinatario antes de confirmar la operación. La presentación desglosada del `TransactionPayload`, especialmente el campo de `confianza` expresado como porcentaje, permitió a los usuarios evaluar la certeza del sistema y decidir de manera informada si confirmar o reintentar el comando.

La figura 4.4 muestra la pantalla de confirmación de una transferencia detectada.

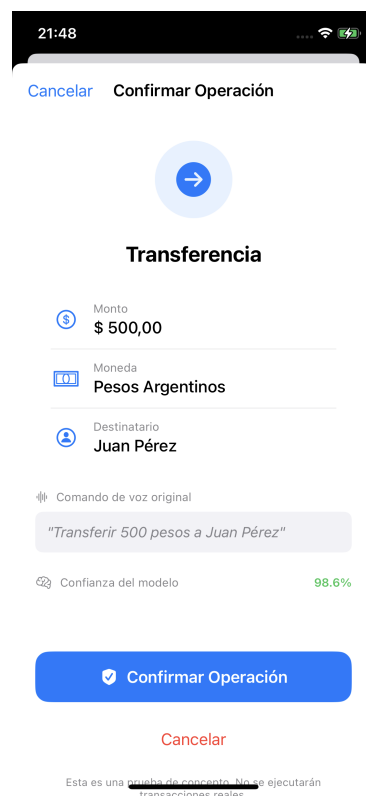


FIGURA 4.4. Pantalla de confirmación para una transferencia exitosa. Se muestra el monto (\$ 500,00), la moneda (Pesos Argentinos), el destinatario (Juan Pérez), la transcripción original y la confianza del modelo (98,6 %).

La figura 4.5 muestra la pantalla de confirmación de una consulta de saldo.

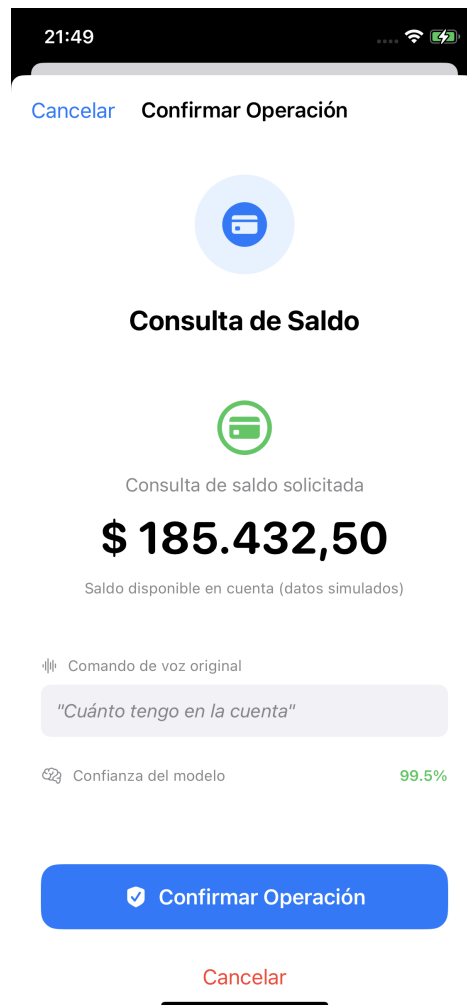


FIGURA 4.5. Confirmación de consulta de saldo. Se presenta el saldo simulado y la confianza del modelo (99,5 %).

La figura 4.6 muestra la pantalla de confirmación de una anulación o cancelación de operación.



FIGURA 4.6. Confirmación de intención de cancelación. El modelo clasificó el comando con una confianza de 96,1 %.

Latencia percibida

El tiempo transcurrido entre el fin del habla y la aparición de la pantalla de confirmación fue percibido como instantáneo en la mayoría de las tareas. Esto es consistente con las mediciones de latencia reportadas en la sección 4.2.2: el procesamiento NLU se completa en el orden de los milisegundos, y la latencia dominante corresponde al *Speech Framework*, que opera en modo *streaming* y entrega la transcripción final con un retardo del orden de 100–300 ms tras el último *buffer* de audio.

Tarea ambigua (T6)

La tarea T6 (anular la transferencia de ayer) fue incluida deliberadamente para evaluar el comportamiento del sistema ante un comando fuera del espacio de capacidades soportadas. La referencia temporal de *ayer* no es una entidad modelada, y la mezcla de *anular* con *transferencia* introduce ambigüedad entre las intenciones CANCELAR y TRANSFERENCIA. El comportamiento observado fue que el modelo clasificó el comando con una confianza inferior al

umbral de 0,65, asignando la intención .desconocida. La aplicación base presentó un mensaje indicando que no pudo interpretar el comando, invitando al usuario a reformularlo. Este comportamiento es el deseado: el sistema prefiere rechazar un comando ambiguo antes que ejecutar una operación financiera potencialmente incorrecta.

En la siguiente tabla 4.8 se ven reflejados los resultados de las pruebas realizadas:

TABLA 4.8. Resultados de las pruebas de usabilidad por tarea. La tasa de éxito se calcula sobre el total de intentos. El tiempo de interacción incluye el habla del usuario, el procesamiento y la confirmación.

Tarea	Tasa de éxito	Tiempo medio	Errores	Intentos
T1 — Transf. estándar	100 %	4,4 s	0,0	1,0
T2 — Transf. coloquial	80 %	3,7 s	0,6	1,8
T3 — Transf. dólares	100 %	3,9 s	0,2	2,4
T4 — Consulta de saldo	100 %	3,3 s	0,0	1,0
T5 — Cancelación	100 %	5,0 s	0,0	3,2
T6 — Comando ambiguo	0 %	4,9 s	3,4	3,4
Promedio	80 %	4,2 s	0,7	—

4.4. Análisis de resultados

Esta sección integra los resultados experimentales presentados en las secciones anteriores, los contrasta con el marco teórico establecido en el capítulo 2, y discute las implicancias para la viabilidad técnica de la solución propuesta.

4.4.1. Validación de la hipótesis central

La hipótesis central de este trabajo sostiene que es posible resolver, de forma íntegra en el dispositivo móvil, el problema dual de clasificar intenciones y extraer entidades nombradas a partir de comandos de voz financieros, utilizando exclusivamente los *frameworks* nativos de Apple: Create ML para el entrenamiento de modelos (MaxEnt para clasificación, CRF para etiquetado de secuencias) y los *frameworks* NaturalLanguage y Core ML para la inferencia local.

Los resultados obtenidos respaldan esta hipótesis. El clasificador de intenciones alcanzó un Macro F1-Score de 1,00 sobre el conjunto de test, un valor que indica una capacidad discriminativa alta para las tres clases definidas. El extractor de entidades logró un Entity F1 *strict* macro de 0,9369, que si bien es inferior al del clasificador, lo que es esperable dado que la extracción de entidades es intrínsecamente una tarea más compleja que la clasificación de texto, resulta suficiente para la prueba de concepto propuesta.

4.4.2. Comparación con arquitecturas alternativas

La comparativa de arquitecturas presentada en el capítulo teórico estableció una comparación cualitativa entre tres familias de arquitecturas: Transformers (BE-TO), redes recurrentes (BiLSTM) y *frameworks* nativos de Apple (Create ML).

Los resultados experimentales permiten ahora complementar esa comparación con evidencia cuantitativa.

En términos de precisión predictiva, la literatura reporta que modelos basados en BETO alcanzan F1-Scores del orden de 0,95–0,98 para clasificación de intenciones en español [28], y que las arquitecturas BiLSTM-CRF obtienen valores de 0,88–0,93 para NER en dominios específicos [27]. Los valores obtenidos con Create ML se ubican en un rango competitivo con los reportados para BiLSTM, aunque ligeramente inferiores a los de BETO.

Sin embargo, la comparación de precisión aislada es insuficiente. El análisis debe ponderarse con las restricciones operativas del *edge computing*:

- **Tamaño del modelo:** los modelos Create ML ocupan menos de 1 MB en disco, frente a los ~440 MB de BETO y los ~5–15 MB de un BiLSTM. Para un dispositivo móvil con almacenamiento limitado y donde la aplicación bancaria compite por espacio con otras aplicaciones, esta diferencia de dos a tres órdenes de magnitud es determinante.
- **Latencia de inferencia:** los modelos nativos resolvieron el *pipeline* completo en menos de 1 ms promedio. Un BETO sin comprimir requiere típicamente más de 200 ms por inferencia en un dispositivo móvil [51], lo que acumularía una latencia perceptible al ejecutar las dos inferencias (clasificación + NER) en secuencia.
- **Integración nativa:** los modelos Create ML no requieren conversión con `coremltools` ni adaptaciones de formato. Se entrenan en macOS, se generan como `.mlmodel`, y Xcode los compila automáticamente a `.mlmodelc` al integrarlos al proyecto. Esta cadena de herramientas cerrada elimina una fuente significativa de errores de integración y mantenimiento.
- **Privacidad nativa:** al operar exclusivamente con frameworks de Apple, el sistema hereda las garantías de privacidad del ecosistema iOS. No se requiere configuración adicional para asegurar que los datos no abandonen el dispositivo, a diferencia de las arquitecturas basadas en PyTorch o TensorFlow que requieren validación explícita de que la inferencia se ejecuta localmente.

En síntesis, la pérdida marginal de precisión respecto a BETO se compensa con ventajas sustanciales en tamaño, latencia, facilidad de integración y garantías de privacidad. Para el caso de uso específico de una prueba de concepto en el sector financiero argentino, donde la privacidad y la latencia son requisitos no negociables, el *trade-off* resulta favorable a la arquitectura nativa.

4.4.3. Rol del `TextNormalizer`

Los ensayos de robustez (tabla 4.5) evidencian el impacto crítico del componente `TextNormalizer` en el rendimiento del sistema. Las categorías de coloquialismos y expresiones numéricas presentan una degradación moderada respecto a la línea base, lo que indica que el normalizador logra convertir una proporción significativa de las variaciones léxicas a formas canónicas que los modelos pueden interpretar.

No obstante, la categoría de mezcla de variaciones, donde confluyen coloquialismos, errores de STT y expresiones numéricas, exhibe la mayor caída de rendimiento. Esto sugiere que el normalizador basado en reglas determinísticas (diccionarios de equivalencias y patrones de expresiones regulares) alcanza un techo de capacidad cuando las variaciones se acumulan. Un enfoque más robusto podría complementar las reglas con un modelo de corrección ortográfica o un *spell checker* adaptado al dominio financiero, capaz de recuperar tokens distorsionados por errores de transcripción.

4.4.4. Limitaciones del estudio

Es pertinente explicitar las limitaciones del presente trabajo, tanto para contextualizar adecuadamente los resultados como para orientar el trabajo futuro:

1. Corpus sintético: los modelos fueron entrenados y evaluados sobre un corpus generado mediante plantillas y aumentación de datos. Si bien esta estrategia permite controlar la distribución de clases y entidades, no captura la totalidad de la variabilidad lingüística del habla espontánea. Un corpus recolectado a partir de grabaciones de usuarios reales proporcionaría una evaluación más representativa.
2. Dominio acotado: el sistema soporta únicamente tres intenciones y tres tipos de entidades. Una aplicación bancaria productiva requeriría un catálogo significativamente más amplio de operaciones (pagos de servicios, inversiones, consultas de movimientos, etc.) y entidades (fechas, números de cuenta, CBU/CVU).
3. Idioma único: la implementación se centra exclusivamente en español rioplatense. Si bien el reconocedor de voz utiliza un mecanismo de *fallback* que selecciona automáticamente el locale con soporte *on-device* (es-MX o es-ES), los modelos NLU y el normalizador fueron entrenados y configurados para el léxico rioplatense. La generalización a otros dialectos del español o a otros idiomas requeriría corpus de entrenamiento adicionales y la validación de que el `TextNormalizer` es extensible a otros sistemas léxicos coloquiales.
4. Ausencia de *backend* real: la aplicación base simula la confirmación de la transacción sin conectarse a un sistema bancario real. La integración con una API transaccional introduciría latencias de red, requisitos de autenticación biométrica (Face ID, Touch ID) y manejo de errores de conectividad que están fuera del alcance de esta prueba de concepto.
5. Mediciones de latencia en macOS: el benchmark de latencia se ejecutó sobre la CPU del equipo de desarrollo (macOS), no sobre un dispositivo iOS físico. Si bien los valores obtenidos ya se encuentran por debajo del umbral de percepción, una medición en dispositivo con Xcode Instruments proporcionaría datos más representativos del rendimiento en producción.
6. Pruebas de usabilidad limitadas: el protocolo de usabilidad se diseñó como evaluación exploratoria con un número reducido de participantes. Un estudio de usabilidad riguroso requeriría una muestra más amplia, diversificada en edad, familiaridad con tecnología de voz y dialecto regional.

4.4.5. Conexión con CRISP-DM

El desarrollo de este trabajo siguió la metodología CRISP-DM (*Cross Industry Standard Process for Data Mining*) como marco organizativo del ciclo de vida del proyecto de *machine learning*. Los resultados obtenidos en esta fase de evaluación retroalimentan el ciclo y permiten identificar las intervenciones prioritarias para una eventual iteración:

- Comprensión de los datos: los errores de la entidad DESTINATARIO señalan la necesidad de enriquecer el corpus con mayor variabilidad de nombres propios, incluyendo apodos, sobrenombres y nombres compuestos más largos.
- Preparación de los datos: la degradación ante errores de STT sugiere incorporar *noise injection* al proceso de aumentación: generar variantes con errores tipográficos y fonéticos simulados para que los modelos desarrollen tolerancia a tokens ruidosos.
- Modelado: si las iteraciones de datos no alcanzan los umbrales deseados, podría evaluarse la migración del clasificador de MaxEnt a *Transfer Learning* con `.dynamicEmbedding`, que aprovecha embeddings contextuales preentrenados por Apple a costa de un incremento moderado en el tamaño del modelo y el tiempo de entrenamiento.
- Despliegue: la validación positiva de la latencia y la privacidad *on-device* confirma la viabilidad técnica del despliegue en dispositivos iOS reales, estableciendo la base para una fase piloto con usuarios del sector financiero.

La tabla 4.9 condensa los indicadores clave de rendimiento de los modelos Core ML del SDK Aurum.

TABLA 4.9. Resumen de rendimiento de los modelos Core ML del SDK Aurum.

Métrica	Clasificador (MaxEnt)	Extractor NER (CRF)
Accuracy / Token Accuracy	100,00 %	89,65 %
Macro F1-Score	1,0000	0,9491
Entity F1 (strict)	—	0,9369
Entity F1 (relaxed)	—	0,9369
Tamaño del modelo	3,1 KB	407,2 KB
Latencia media de inferencia	0,02 ms	0,26 ms

Capítulo 5

Conclusiones

En este capítulo se presentan las conclusiones generales derivadas del desarrollo de la prueba de concepto y se proponen líneas de trabajo futuro que permitirían evolucionar el prototipo hacia un producto integrable en un entorno bancario productivo.

5.1. Resultados generales

El presente trabajo demostró la viabilidad técnica de implementar un sistema de transacciones financieras por voz que opera de forma completamente local en dispositivos móviles iOS. A continuación se enumeran los principales logros alcanzados:

- Se diseñó e implementó el SDK Aurum como *framework* nativo reutilizable, que encapsula la totalidad del *pipeline* de procesamiento de voz: captura de audio, transcripción, normalización e inferencia de modelos de *machine learning*. Su arquitectura basada en el patrón de delegación permite que cualquier aplicación bancaria lo integre implementando un único protocolo público.
- Se entrenaron dos modelos de *machine learning* con Create ML (MaxEnt para clasificación de intenciones y CRF para extracción de entidades) que alcanzaron un desempeño satisfactorio para la prueba de concepto: 100 % de *accuracy* en la clasificación de intenciones y un F1 *strict* de 0,9369 en la extracción de entidades, con tamaños de modelo de 3,1 KB y 407,2 KB respectivamente.
- Se validó que el procesamiento estrictamente local (*on-device*) es factible en el hardware actual de Apple. Los modelos Core ML resolvieron el *pipeline* completo de inferencia con latencias del orden de fracciones de milisegundo, muy por debajo del umbral de percepción humana de 100 ms establecido por Nielsen.
- Se garantizó la privacidad por diseño: en ningún momento del flujo los datos de audio ni las transcripciones abandonan el dispositivo físico, lo que satisface las restricciones de confidencialidad exigidas por el sector financiero y se alinea con la Ley argentina de Protección de Datos Personales (Ley 25.326).
- Se desarrolló una aplicación base funcional con SwiftUI que valida la integración con el SDK y demuestra el flujo completo de interacción: desde la

activación del micrófono hasta la presentación del *payload* estructurado en una pantalla de confirmación.

- La metodología CRISP-DM resultó adecuada para estructurar las fases del proyecto, lo que permitió iterar entre la comprensión del dominio financiero, la generación del *dataset* sintético, el entrenamiento de los modelos y la evaluación de resultados.

En cuanto a las dificultades encontradas, la principal fue la ausencia de datos reales de comandos de voz bancarios, resuelta mediante la generación de un corpus sintético con técnicas de aumentación de datos. Adicionalmente, se identificó que el locale `es-AR` no dispone de modelo de reconocimiento de voz *on-device* en la mayoría de los dispositivos iOS, lo que requirió implementar un mecanismo de selección automática de locale con *fallback* a `es-MX`.

En síntesis, los resultados obtenidos validan que la convergencia entre los *frameworks* nativos de Apple (Speech, Core ML, NaturalLanguage) y las técnicas de procesamiento de lenguaje natural permite construir soluciones de voz robustas, privadas y de baja latencia para el dominio financiero, sin dependencia de servicios en la nube.

5.2. Próximos pasos

El presente trabajo establece las bases técnicas para una solución de voz bancaria *on-device*. A continuación se proponen las líneas de trabajo futuro que permitirían acercar el prototipo a un nivel de madurez productivo:

- Validación con usuarios reales: ejecutar un estudio de usabilidad con una muestra amplia y diversificada en edad, dialecto regional y familiaridad con tecnología de voz, que permita obtener métricas estadísticamente significativas de tasa de éxito, tiempo de interacción y satisfacción.
- Ampliación del corpus de entrenamiento: incorporar grabaciones de voz reales (con consentimiento informado) para entrenar los modelos con datos representativos del habla natural, incluyendo ruido ambiental, variaciones prosódicas y errores de dicción.
- Expansión del dominio transaccional: extender las intenciones soportadas a operaciones como pagos de servicios, consultas de últimos movimientos, inversiones y operaciones con tarjetas, lo que implicaría ampliar el esquema de entidades y reentrenar los modelos.
- Mediciones en dispositivo físico: ejecutar *benchmarks* de latencia con Xcode Instruments sobre dispositivos iPhone con Neural Engine, lo que proporcionaría datos de rendimiento representativos del entorno de producción.
- Integración con biometría de voz: investigar la factibilidad de incorporar un módulo de verificación de identidad basado en las características acústicas del hablante (*speaker verification*), que operaría como factor adicional de autenticación previo a la ejecución de la transacción.
- Soporte multilingüe: adaptar el *pipeline* para soportar otros idiomas y variantes regionales del español, aprovechando la arquitectura modular del SDK que permite intercambiar modelos sin modificar la lógica de orquestación.

- Evaluación de seguridad adversarial: analizar la robustez del sistema ante ataques de inyección acústica (*adversarial audio attacks*) y definir contramedidas, como umbrales adaptativos de confianza y detección de anomalías en la señal de entrada.
- Integración con el *core* bancario: diseñar la capa de comunicación segura entre el SDK y los servicios *backend* del banco, incluyendo la firma criptográfica del *payload*, la autenticación mutua TLS y la trazabilidad de las operaciones para auditoría.

Bibliografía

- [1] Ángeles Núñez y Laura Núñez Letamendia. «La transformación digital de la banca: innovación, colaboración y sostenibilidad.» En: *Harvard Deusto* (2024).
- [2] Informe económico y financiero. «El momento de la inteligencia artificial». En: *ESADE* (2023).
- [3] Leticia Rengifo. *Inclusión financiera en Argentina y el efecto Pandemia*. Visitado el 2026-03-25. 2022. URL: <https://repositorio.utdt.edu/items/c8641051-5953-4145-a6ce-f98147aa7093>.
- [4] Banco Central de la República Argentina. *Informe de Inclusión Financiera - Segundo semestre 2022*. <https://www.bcra.gob.ar/publicaciones/informe-de-inclusion-financiera-segundo-semestre-2022/>. Dic. de 2022. (Visitado 02-05-2023).
- [5] Diario Clarín. *El Galicia apuesta a seguir siendo el mayor banco privado de la Argentina*. Visitado el 2026-03-28. 2024. URL: https://www.clarin.com/80-aniversario/bancos-finanzas/galicia-apuesta-seguir-mayor-banco-privado-argentina_0_iVSN8H9vt1.html.
- [6] Frontline VC. *Voice AI and Financial Services: Introducing Avallon*. Visitado el 2026-03-29. 2025. URL: <https://frontline.vc/blog/voice-ai-and-financial-services-introducing-avallon/>.
- [7] Deepgram. *What Is a Voice AI Agent? The Complete Guide for 2026*. Visitado el 2026-03-29. 2026. URL: <https://deepgram.com/learn/what-is-a-voice-ai-agent-2026>.
- [8] Kardome. *The Voice Privacy Problem: Security and Latency in Cloud-Based Voice Assistants*. Visitado el 2026-03-29. 2025. URL: <https://www.kardome.com/resources/blog/voice-privacy-concerns/>.
- [9] Bank of America. *A Decade of AI Innovation: BofA's Virtual Assistant Erica Surpasses 3 Billion Client Interactions*. Visitado el 2026-03-29. 2025. URL: <https://newsroom.bankofamerica.com/content/newsroom/press-releases/2025/08/a-decade-of-ai-innovation--bofa-s-virtual-assistant-erica-surpas.html>.
- [10] Amazon Web Services. *Capital One Completes Migration from Data Centers to AWS*. Visitado el 2026-03-29. 2023. URL: <https://aws.amazon.com/solutions/case-studies/capital-one-all-in-on-aws/>.
- [11] Apple Inc. *SiriKit: Payments Domain*. Visitado el 2026-03-29. Apple Developer Documentation. 2025. URL: <https://developer.apple.com/documentation/sirikit/payments>.
- [12] Apple Inc. «Apple Intelligence Foundation Language Models». En: *arXiv preprint arXiv:2407.21075* (2024). URL: <https://arxiv.org/abs/2407.21075>.
- [13] Apple Machine Learning Research. *Deploying Transformers on the Apple Neural Engine*. Visitado el 2026-03-29. 2022. URL: <https://machinelearning.apple.com/research/neural-engine-transformers>.

- [14] Apple Inc. *Core ML: Integrate machine learning models into your app*. Visitado el 2026-03-29. Apple Developer Documentation. 2025. URL: <https://developer.apple.com/documentation/coreml>.
- [15] Instituto Nacional de Estadística y Censos. *Estudio Nacional sobre el Perfil de las Personas con Discapacidad (EPPD): Resultados definitivos*. Inf. téc. Visitado el 2026-03-29. INDEC, República Argentina, 2018. URL: <https://www.indec.gov.ar/indec/web/Nivel4-Tema-2-41-135>.
- [16] Organización Mundial de la Salud. *Informe mundial sobre la visión*. Inf. téc. Visitado el 2026-03-29. World Health Organization, 2019. URL: <https://www.who.int/es/publications/i/item/9789241516570>.
- [17] Rüdiger Wirth y Jochen Hipp. «CRISP-DM: Towards a standard process model for data mining». En: *Proceedings of the 4th international conference on the practical applications of knowledge discovery and data mining*. Springer-Verlag. 2000, págs. 29-39.
- [18] Qian Chen, Zhu Zhuo y Wen Wang. «BERT for joint intent prediction and slot filling». En: *arXiv preprint arXiv:1902.10909* (2019).
- [19] Weisong Shi et al. «Edge Computing: Vision and Challenges». En: *IEEE Internet of Things Journal* 3.5 (2016), págs. 637-646.
- [20] Ann Cavoukian. «Privacy by Design: The 7 Foundational Principles». En: *Information and Privacy Commissioner of Ontario, Canada* 5 (2009), págs. 1-12.
- [21] Apple Inc. *The Swift Programming Language: Language Guide*. Visitado el 2026-04-03. Apple Developer Documentation. 2026. URL: <https://docs.swift.org/swift-book/>.
- [22] Apple Inc. *Xcode: Integrated Development Environment*. Visitado el 2026-04-02. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/xcode/>.
- [23] Apple Inc. *AVFoundation: Work with audiovisual assets, control device cameras, process audio, and configure system audio interactions*. Visitado el 2026-04-02. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/avfoundation>.
- [24] Apple Inc. *Speech: Perform speech recognition on recorded or live audio*. Visitado el 2026-04-03. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/speech>.
- [25] Apple Inc. *Core ML: Integrate machine learning models into your app*. Visitado el 2026-04-02. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/coreml>.
- [26] Apple Inc. *coremltools: Unified conversion API and optimization tools for Core ML*. Visitado el 2026-04-12. Apple Open Source. 2026. URL: <https://coremltools.readme.io/>.
- [27] Guillaume Lample et al. «Neural Architectures for Named Entity Recognition». En: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2016, págs. 260-270.
- [28] José Cañete et al. «Spanish pre-trained bert model and evaluation data». En: *PML4DC at ICLR 2020*. 2020.
- [29] Jeffrey L Elman. «Finding structure in time». En: *Cognitive science* 14.2 (1990), págs. 179-211.
- [30] Sepp Hochreiter y Jürgen Schmidhuber. «Long short-term memory». En: *Neural computation* 9.8 (1997), págs. 1735-1780.

- [31] Kyunghyun Cho et al. «Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation». En: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, págs. 1724-1734.
- [32] Zhiheng Huang, Wei Xu y Kai Yu. «Bidirectional LSTM-CRF models for sequence tagging». En: *arXiv preprint arXiv:1508.01991* (2015).
- [33] Saurabh Dhar et al. «A survey of on-device machine learning: An algorithms and learning theory perspective». En: *ACM Transactions on Internet of Things (TIOT)* 2.3 (2021), págs. 1-49.
- [34] Apple Inc. *Natural Language: Analyze natural language text and deduce its language-specific metadata*. Visitado el 2026-03-29. Apple Developer Documentation. 2025. URL: <https://developer.apple.com/documentation/naturallanguage>.
- [35] Apple Inc. *Creating a Framework: Package dynamic libraries and resources to share code*. Visitado el 2026-04-03. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/xcode/creating-a-framework>.
- [36] Steven Y Feng et al. «A survey of data augmentation approaches for NLP». En: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021* (2021), págs. 968-988.
- [37] Apple Inc. *Encoding and Decoding Custom Types*. Visitado el 2026-04-12. Apple Developer Documentation. 2026. URL: https://developer.apple.com/documentation/foundation/archives_and_serialization/encoding_and_decoding_custom_types.
- [38] Tim Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. IETF, 2017. URL: <https://datatracker.ietf.org/doc/html/rfc8259>.
- [39] Adam L Berger, Stephen A Della Pietra y Vincent J Della Pietra. «A maximum entropy approach to natural language processing». En: *Computational Linguistics* 22.1 (1996), págs. 39-71.
- [40] John D Lafferty, Andrew McCallum y Fernando C N Pereira. «Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data». En: *Proceedings of the Eighteenth International Conference on Machine Learning (ICML)*. Morgan Kaufmann, 2001, págs. 282-289.
- [41] Andrew Viterbi. «Error bounds for convolutional codes and an asymptotically optimum decoding algorithm». En: *IEEE Transactions on Information Theory* 13.2 (1967), págs. 260-269.
- [42] Lance A Ramshaw y Mitchell P Marcus. «Text chunking using transformation-based learning». En: *Third Workshop on Very Large Corpora* (1995).
- [43] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [44] Apple Inc. *Natural Language Framework Documentation*. 2024. URL: <https://developer.apple.com/documentation/naturallanguage>.
- [45] Apple Inc. *Using Delegates to Customize Object Behavior*. 2024. URL: <https://developer.apple.com/documentation/swift/using-delegates-to-customize-object-behavior>.
- [46] Apple Inc. *SwiftUI Documentation*. 2024. URL: <https://developer.apple.com/documentation/swiftui>.
- [47] Jakob Nielsen. *Confirmation Dialogs Can Prevent User Errors*. 2018. URL: <https://www.nngroup.com/articles/confirmation-dialog/>.

- [48] Apple Inc. *Human Interface Guidelines: Accessing User Data*. 2024. URL: <https://developer.apple.com/design/human-interface-guidelines/accessing-private-data>.
- [49] Ann Cavoukian. «Privacy by Design: The 7 Foundational Principles». En: *Information and Privacy Commissioner of Ontario* (2009).
- [50] Jakob Nielsen. *Response Times: The 3 Important Limits*. 1993. URL: <https://www.nngroup.com/articles/response-times-3-important-limits/>.
- [51] Zhiqing Sun et al. «MobileBERT: a Compact Task-Agnostic BERT for Resource-Limited Devices». En: *ACL*. 2020.